LAB MANUAL

AI TOOLS LAB

VR-23

III B. Tech II Sem

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VIGNAN'S INSTITUTE OF INFORMATION TECHNOLOGY (A)

(Approved by AICTE & Affiliated to JNTU-GV) Estd. – 2002

NAAC, NBA Accredited & ISO 9001:2008, ISO14001:2004, OHSAS 18001:2007 Certified Institution

Beside VSEZ, Duvvada, Visakhapatnam-530049. A.P.

Phone : 0891- 2755222 / 333 / 444 :: Fax : 0891 - 2752333 :: E-Mail : vignaniit@yahoo.com,

www.vignaniit.com

SYLLABUS

III Year - II Semester	AI TOOLS LAB	L	T	P	C
1005233210		0	0	3	1.5

COURSE OBJECTIVES:

This course is introduced to

1. Learn the fundamentals of most widely used Python packages NumPy, Pandas and Matpotlib, and then apply them to Data Analysis and Data Visualization projects.
2. To introduce the fundamental techniques and principles of Neural Networks
3. Teach students the leading trends and systems in natural language processing

COURSE OUTCOMES:

CO's	Course Outcomes	BTL
CO1	Apply the tools of AI in the field of Engineering.	L3
CO2	Identify the deep learning algorithms which are more appropriate for various types of learning tasks in various domains.	L2
CO3	Design and implement solutions to classification, regression, and clustering problems	L4
CO4	Implement deep learning algorithms and solve real-world problems	L3
CO5	Understand machine learning techniques used in NLP, including hidden Markov models and probabilistic context-free grammars, clustering and unsupervised methods	L2

LIST OF EXPERIMENTS

S. No	Experiments	CO	BTL
1	**Numpy**: Illustrate the concepts multi-dimensional arrays and matrices, along with a large library of high-level mathematical functions to operate on these arrays using numpy	CO1	L3
2	**Pandas:** Visualize New York Motor Vehicle Crash Data Using Python, Pandas, and Matplotlib. **Datasets Details:** https://data.ny.gov/Transportation/Motor-Vehicle-Crashes-Case-Information-Three-Year-/e8ky-4vqe https://data.ny.gov/Transportation/Motor-Vehicle-Crashes-Individual-Information-Three/ir4y-sesj	CO3	L4

S. No	Experiments	CO	BTL
	https://data.ny.gov/Transportation/Motor-Vehicle-Crashes-Violation-Information-Three-/abfj-y7uq https://data.ny.gov/Transportation/Motor-Vehicle-Crashes-Vehicle-Information-Three-Ye/xe9x- a24f		
3	**Tensor-Flow:** Learn simple data curation by creating a pickle with formatted datasets for training, development and testing in Tensor Flow and develop visualizations in tensor board.	CO3	L4
4	Create convolutional neural networks in Tensor Flow.	CO4	L3
5	**Image recognition** (or image classification) : identifying images and categorizing them in one of several predefined distinct classes using neural network models.	CO2	L2
6	**OpenCV:** Develop an online writing Whiteboard with minimal features for online classes	CO1	L3
7	**Keras:** Recognize handwritten digits from MNIST using Keras	CO4	L3
8	**Scikit-learn: Write** a Python program using Scikit-learn to split the iris dataset into 70% train data and 30% test data. Out of total 150 records, the training set will contain 120 records and the test set contains 30 of those records. Print both datasets	CO3	L3
9	Design a perceptron classifier to classify handwritten numerical digits (0-9). Implement using Scikit or Weka.	CO3	L4
10	**NLP:** Program to illustrate the concepts sentence segmentation, word tokenization, stemming and lemmatization, Hidden markov model (HMM) for Parts of speech (PoS)tag.	CO5	L2

INDEX

1. **Numpy: Illustrate the concepts multi-dimensional arrays and matrices, along with a large library of high-level mathematical functions to operate on these arrays using numpy**

Introduction:

NumPy (Numerical Python) is a library for scientific computing that supports:

- Multi-dimensional array objects (ndarray)
- Mathematical, logical, and statistical operations on arrays
- Linear algebra and matrix operations

Software Requirements:

- Python (≥3.8)
- NumPy library (pip install numpy)
- Any IDE (Jupyter Notebook / Google Colab / VS Code / PyCharm)

Procedure / Steps to Execute:

Step 1: Import NumPy

```
import numpy as np
print("NumPy version:", np.__version__)
```

Step 2: Create 1D, 2D, and 3D Arrays

```
# 1D Array
a = np.array([1, 2, 3, 4, 5])
# 2D Array
b = np.array([[1, 2, 3],
        [4, 5, 6]])
# 3D Array
c = np.array([[[1, 2], [3, 4]],
        [[5, 6], [7, 8]]])
print("1D Array:\n", a)
print("2D Array:\n", b)
print("3D Array:\n", c)
```

Step 3: Create Arrays

```
# 1D, 2D, and 3D Arrays
a = np.array([1, 2, 3, 4, 5])
```

```
b = np.array([[1, 2, 3],
              [4, 5, 6]])
c = np.array([[[1, 2], [3, 4]],
              [[5, 6], [7, 8]]])
print("1D Array:\n", a)
print("2D Array:\n", b)
print("3D Array:\n", c)
```

Step 4: Array Attributes

```
print("Dimensions of b:", b.ndim)
print("Shape of b:", b.shape)
print("Data type of b:", b.dtype)
print("Total elements:", b.size)
```

Step 5: Indexing and Slicing

```
print("First row of 2D array:", b[0])
print("Element at (1,2):", b[1,2])
print("First two elements of 1D array:", a[:2])
```

Step 6: Matrix Operations

```
x = np.array([[1, 2], [3, 4]])
y = np.array([[5, 6], [7, 8]])
print("Matrix Addition:\n", x + y)
print("Matrix Subtraction:\n", x - y)
print("Matrix Multiplication:\n", np.dot(x, y))
print("Transpose of x:\n", x.T)
print("Inverse of x:\n", np.linalg.inv(x))
```

Step 7: Mathematical Functions

```
arr = np.array([1, 4, 9, 16, 25])
print("Square roots:", np.sqrt(arr))
```

print("Exponential:", np.exp(arr))

print("Sine values:", np.sin(arr))

print("Mean:", np.mean(arr))

print("Standard Deviation:", np.std(arr))

OUTPUT:

In [1]:
```
import numpy as np
print("NumPy version:", np.__version__)
```

```
NumPy version: 1.24.3
```

In [2]:
```
# 1D, 2D, and 3D Arrays
a = np.array([1, 2, 3, 4, 5])
b = np.array([[1, 2, 3],
              [4, 5, 6]])
c = np.array([[[1, 2], [3, 4]],
              [[5, 6], [7, 8]]])

print("1D Array:\n", a)
print("2D Array:\n", b)
print("3D Array:\n", c)
```

```
1D Array:
 [1 2 3 4 5]
2D Array:
 [[1 2 3]
 [4 5 6]]
3D Array:
 [[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]
```

In [3]:
```
print("Dimensions of b:", b.ndim)
print("Shape of b:", b.shape)
print("Data type of b:", b.dtype)
print("Total elements:", b.size)
```

```
Dimensions of b: 2
Shape of b: (2, 3)
Data type of b: int32
Total elements: 6
```

In [4]:
```
print("First row of 2D array:", b[0])
print("Element at (1,2):", b[1,2])
print("First two elements of 1D array:", a[:2])
```

```
First row of 2D array: [1 2 3]
Element at (1,2): 6
First two elements of 1D array: [1 2]
```

In [5]:
```
x = np.array([[1, 2], [3, 4]])
y = np.array([[5, 6], [7, 8]])

print("Matrix Addition:\n", x + y)
print("Matrix Subtraction:\n", x - y)
print("Matrix Multiplication:\n", np.dot(x, y))
print("Transpose of x:\n", x.T)
print("Inverse of x:\n", np.linalg.inv(x))
```

```
Matrix Addition:
 [[ 6  8]
 [10 12]]
Matrix Subtraction:
 [[-4 -4]
 [-4 -4]]
Matrix Multiplication:
 [[19 22]
 [43 50]]
Transpose of x:
 [[1 3]
 [2 4]]
Inverse of x:
 [[-2.   1. ]
 [ 1.5 -0.5]]
```

In [6]:
```
arr = np.array([1, 4, 9, 16, 25])

print("Square roots:", np.sqrt(arr))
print("Exponential:", np.exp(arr))
print("Sine values:", np.sin(arr))
print("Mean:", np.mean(arr))
print("Standard Deviation:", np.std(arr))
```

```
Square roots: [1. 2. 3. 4. 5.]
Exponential: [2.71828183e+00 5.45981500e+01 8.10308393e+03 8.88611052e+06
 7.20048993e+10]
Sine values: [ 0.84147098 -0.7568025   0.41211849 -0.28790332 -0.13235175]
Mean: 11.0
Standard Deviation: 8.648699324175862
```

2. Pandas: Visualize New York Motor Vehicle Crash Data using Python, Pandas, and Matplotlib

Datasets Details:

https://data.ny.gov/Transportation/Motor-Vehicle-Crashes-Case-Information-Three-Year-/e8ky-4vqe

https://data.ny.gov/Transportation/Motor-Vehicle-Crashes-Individual-Information-Three/ir4y-sesj

https://data.ny.gov/Transportation/Motor-Vehicle-Crashes-Violation-Information-Three-/abfj-y7uq

https://data.ny.gov/Transportation/Motor-Vehicle-Crashes-Vehicle-Information-Three-Ye/xe9x- a24f

Introduction:

Python is a widely used high-level programming language known for its simplicity and strong support for data analysis. Pandas is a Python library that provides powerful tools to load, clean, manipulate, and analyze structured data using DataFrames. Matplotlib is a popular visualization library used to create a wide range of charts and graphs to understand data patterns clearly.

Software Required:

- Python (3.8 or later)
- Libraries: pandas, matplotlib
  (Install using pip install pandas matplotlib)
- Jupyter Notebook

Procedure:

Step 1: Import Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
```

Step 2: Load Dataset

```
URL_CASE  = "https://data.ny.gov/api/views/e8ky-4vqe/rows.csv?accessType=DOWNLOAD"
URL_INDIV = "https://data.ny.gov/api/views/ir4y-sesj/rows.csv?accessType=DOWNLOAD"
URL_VIOL  = "https://data.ny.gov/api/views/abfj-y7uq/rows.csv?accessType=DOWNLOAD"
URL_VEH   = "https://data.ny.gov/api/views/xe9x-a24f/rows.csv?accessType=DOWNLOAD"
# Load CASE table (crashes)
case = pd.read_csv(
URL_CASE,
low_memory=False,
parse_dates=True)
case.columns = case.columns.str.strip()
# Load other tables (optional for joins later)
```

```
indiv = pd.read_csv(URL_INDIV, low_memory=False)
viol  = pd.read_csv(URL_VIOL,  low_memory=False)
veh   = pd.read_csv(URL_VEH,   low_memory=False)
for df in (indiv, viol, veh):
df.columns = df.columns.str.strip()
case.head(), indiv.head(), viol.head(), veh.head()
```

Step 3: Inspect columns & identify keys

```
print("CASE columns:", case.columns.tolist()[:50])
print("INDIV columns:", indiv.columns.tolist()[:50])
print("VIOL columns:",  viol.columns.tolist()[:50])
print("VEH columns:",   veh.columns.tolist()[:50])
```

Step 4: Normalize key and time/location columns

Step 5: Visualizations

A) Crashes per Month

```
monthly = case.groupby("month").size().rename("crashes").sort_index()
monthly.index = monthly.index.astype(str)  # nicer x-axis labels
plt.figure()
monthly.plot(kind="line", marker="o")
plt.title("Crashes per Month (Last 3 Years)")
plt.xlabel("Month")
plt.ylabel("Number of Crashes")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

B) Top 10 Counties by Total Crashes

```
geo_col = COUNTY if COUNTY else BOROUGH  # prefer COUNTY, fallback to BOROUGH if present
top_geo = case[geo_col].value_counts().head(10)
plt.figure()
top_geo.plot(kind="bar")
plt.title("Top 10 Counties by Crash Count")
plt.xlabel("County")
plt.ylabel("Crashes")
plt.xticks(rotation=45, ha="right")
```

```
plt.tight_layout()
plt.show()
```

C) Crashes by Day of Week

```
plt.figure(figsize=(8, 5))
case['Day of Week'].value_counts().reindex(
["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]).plot(kind='bar',
color='skyblue', edgecolor='black')
plt.title("Crashes by Day of Week")
plt.xlabel("Day")
plt.ylabel("Number of Crashes")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

D) Crashes by Weather Conditions

```
plt.figure(figsize=(9, 5))
case['Weather Conditions'].value_counts().head(10).plot(kind='bar', color='lightgreen',
edgecolor='black')
plt.title("Top 10 Weather Conditions During Crashes")
plt.xlabel("Weather Condition")
plt.ylabel("Crash Count")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

E) Crashes by Lighting Conditions

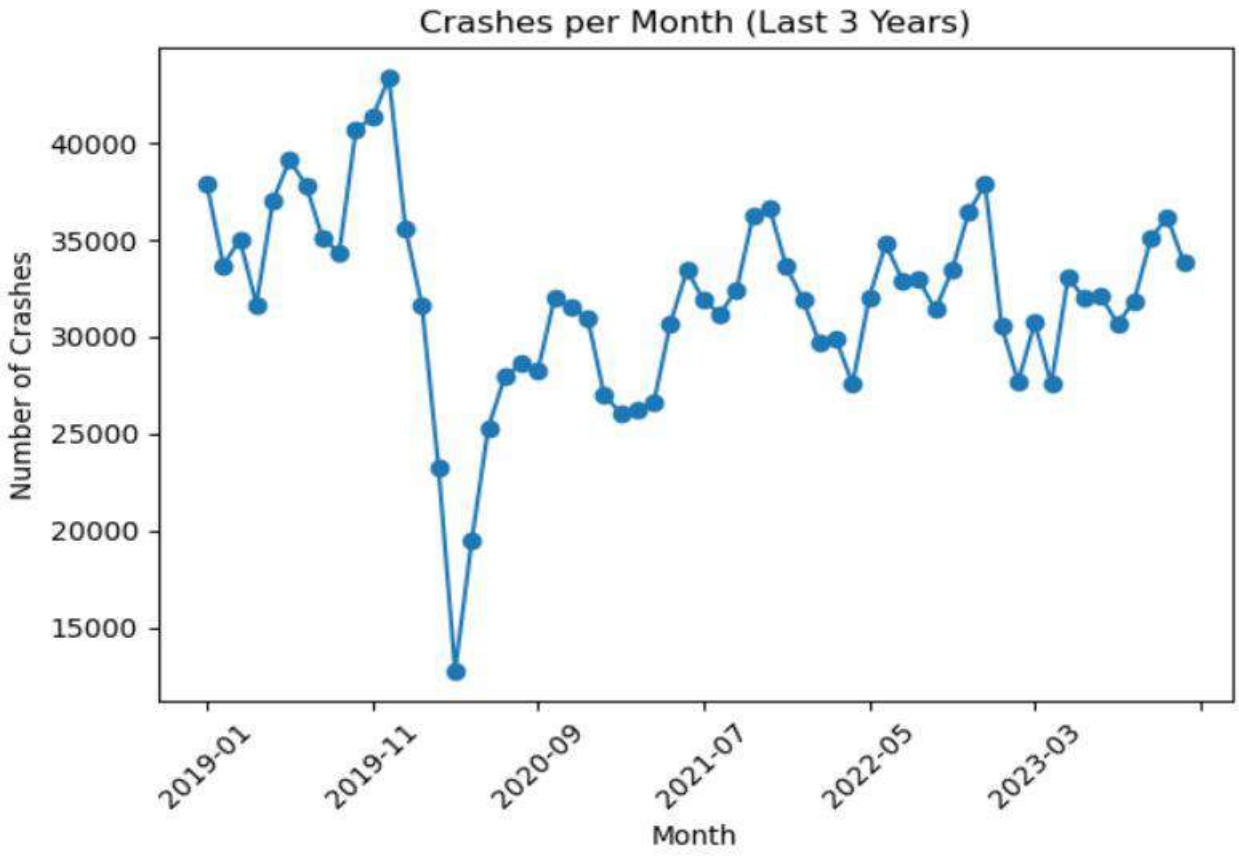
```
plt.figure(figsize=(8, 5))
case['Lighting Conditions'].value_counts().plot(kind='bar', color='gold', edgecolor='black')
plt.title("Crashes by Lighting Conditions")
plt.xlabel("Lighting Condition")
plt.ylabel("Crash Count")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

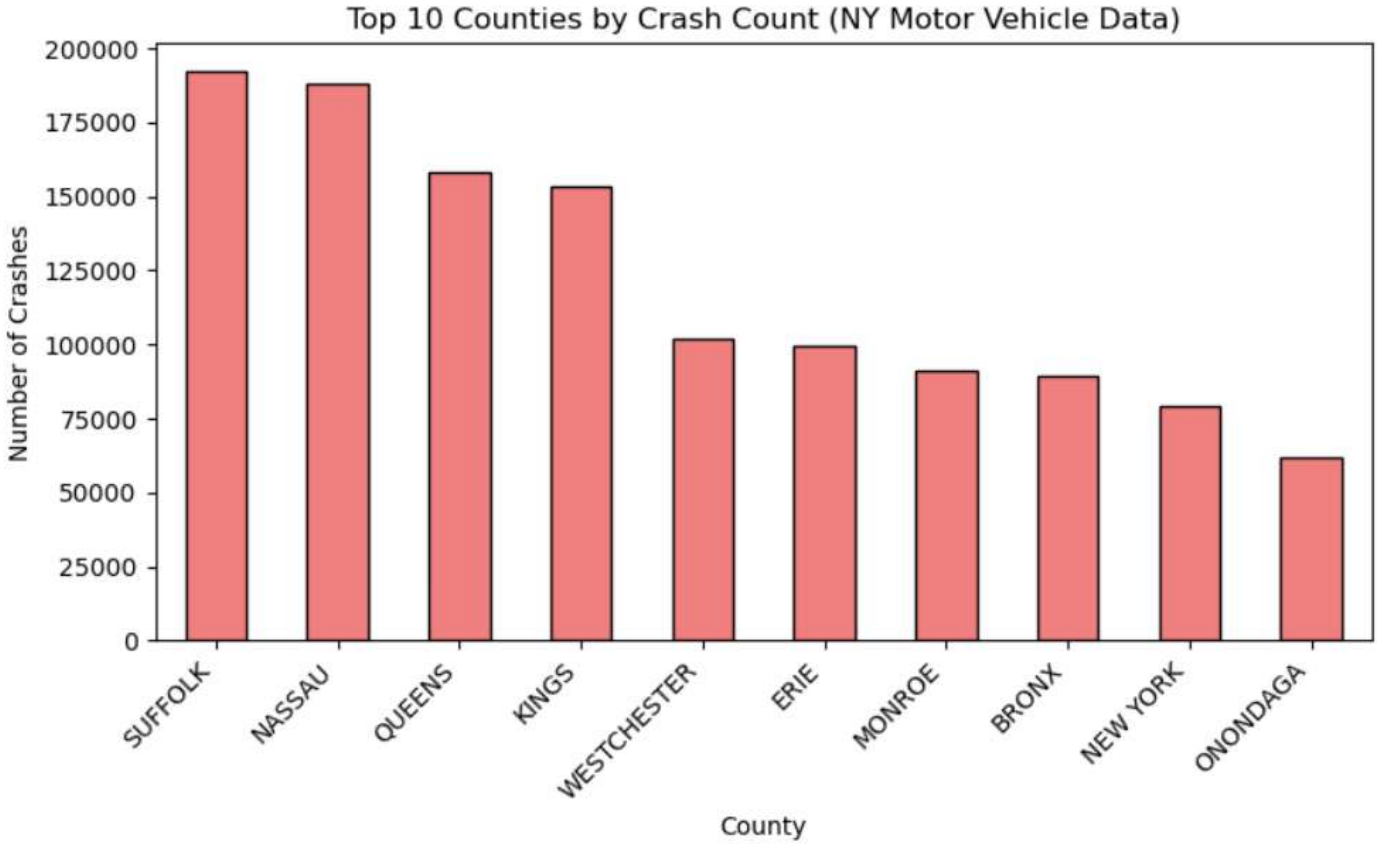
F) Monthly Crash Trend

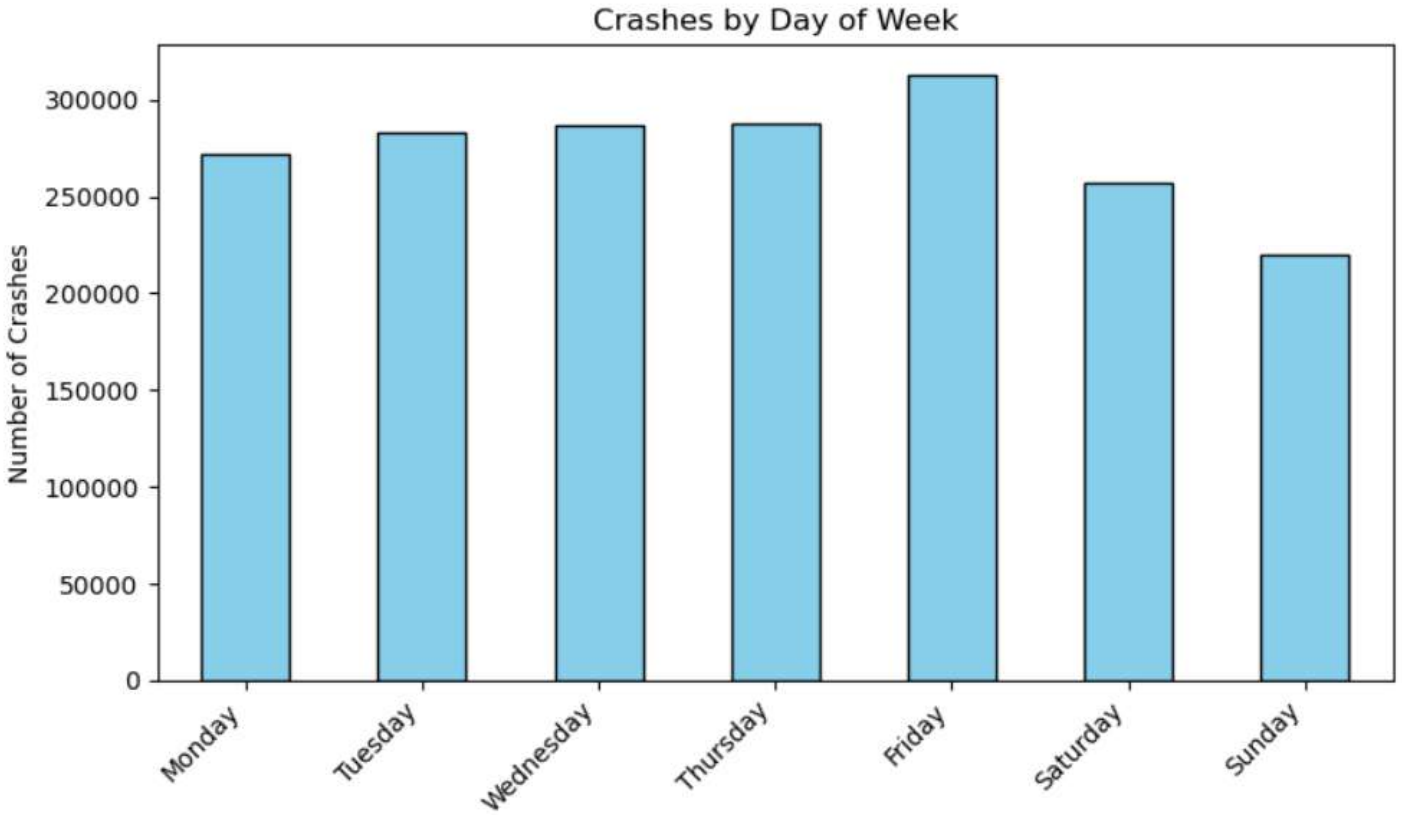
```
case['_date'] = pd.to_datetime(case['Date'], errors='coerce')
monthly_trend = case.groupby(case['_date'].dt.to_period('M')).size()
plt.figure(figsize=(10, 5))
monthly_trend.plot(marker='o', color='royalblue')
plt.title("Monthly Crash Trend (3-Year Period)")
plt.xlabel("Month")
plt.ylabel("Number of Crashes")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

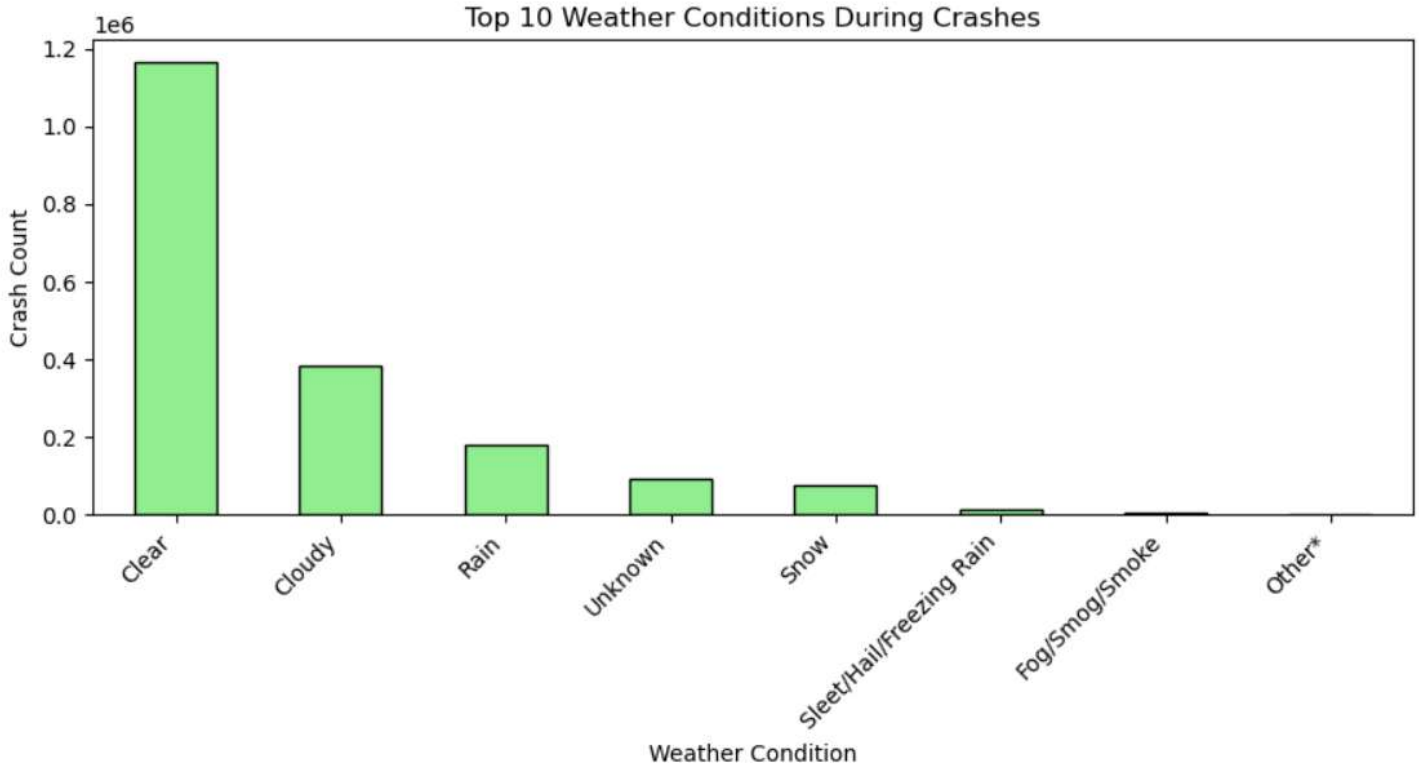
OUTPUT:

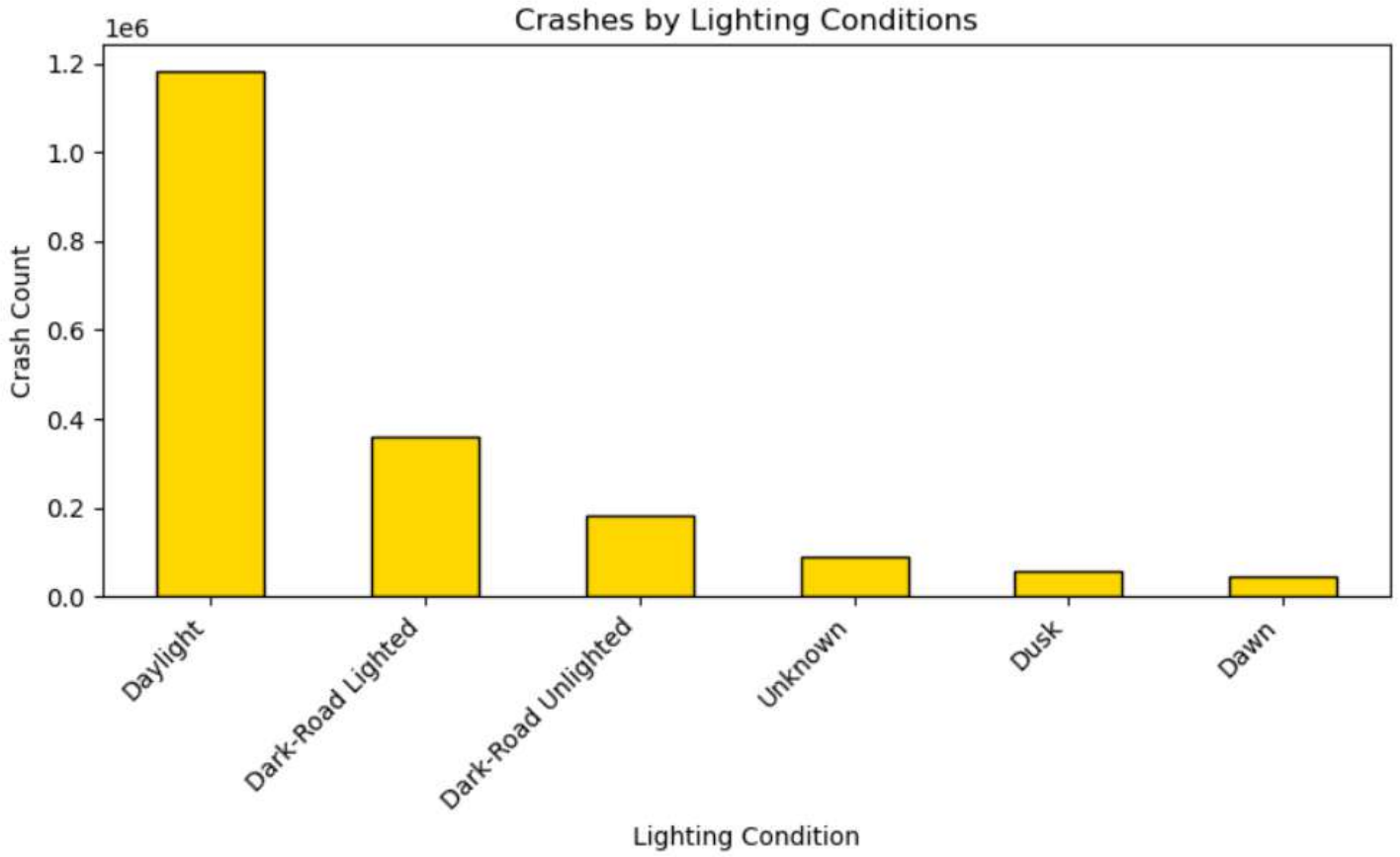
```
Out[2]: (   Year          Crash Descriptor  ...                    Event Descriptor Number of Vehicles Involved
         0  2019  Property Damage Accident  ...   Fence, Collision With Fixed Object                           1
         1  2019  Property Damage Accident  ...                                 Deer                           1
         2  2019           Injury Accident  ...  Other Motor Vehicle, Collision With                           3
         3  2019           Injury Accident  ...  Other Motor Vehicle, Collision With                           2
         4  2019  Property Damage Accident  ...    Tree, Collision With Fixed Object                           1

         [5 rows x 18 columns],
            Year  Case Individual ID  Case Vehicle ID Victim Status  ...  Injury Descriptor Injury Location Injury Severity   Age
         0  2019           20148315         15350252         Shock  ...  Complaint of Pain     Entire Body           Minor  39.0
         1  2019           21577130         16426203   Not Entered  ...        Not Entered     Not Entered       Uninjured  34.0
         2  2019           21577131         16426204   Not Entered  ...        Not Entered     Not Entered       Uninjured  44.0
         3  2019           21577132         16426202   Not Entered  ...        Not Entered     Not Entered       Uninjured  31.0
         4  2019           21577255         16426295   Not Entered  ...        Not Entered     Not Entered       Uninjured  24.0

         [5 rows x 15 columns],
            Year  Violation Description Violation Code  Case Individual ID
         0  2020   FAILED TO KEEP RIGHT          1120A            23485688
         1  2020    UNLICENSED OPERATOR           5091            23485692
         2  2020  FOLLOWING TOO CLOSELY          1129A            23485702
         3  2020   FAILED TO KEEP RIGHT          1120A            23485722
         4  2021    UNLICENSED OPERATOR           5091            24282770,
            Year  Case Vehicle ID Vehicle Body Type  ... Contributing Factor 2 Description      Event Type        Partial VIN
         0  2021         18127660   UNKNOWN VEHICLE  ...                          Unknown     Not Entered                NaN
         1  2019         16565628             SEDAN  ...                      Not Entered     Not Entered                NaN
         2  2019         16565632      4 DOOR SEDAN  ...                  Not Applicable  Not Applicable  1N4AL3AP8HN311852
         3  2019         17094303             SEDAN  ...                      Not Entered     Not Entered                NaN
         4  2019         17094305          SUBURBAN  ...                      Not Entered     Not Entered  3CZRE4H5XAG700793

         [5 rows x 19 columns])
```

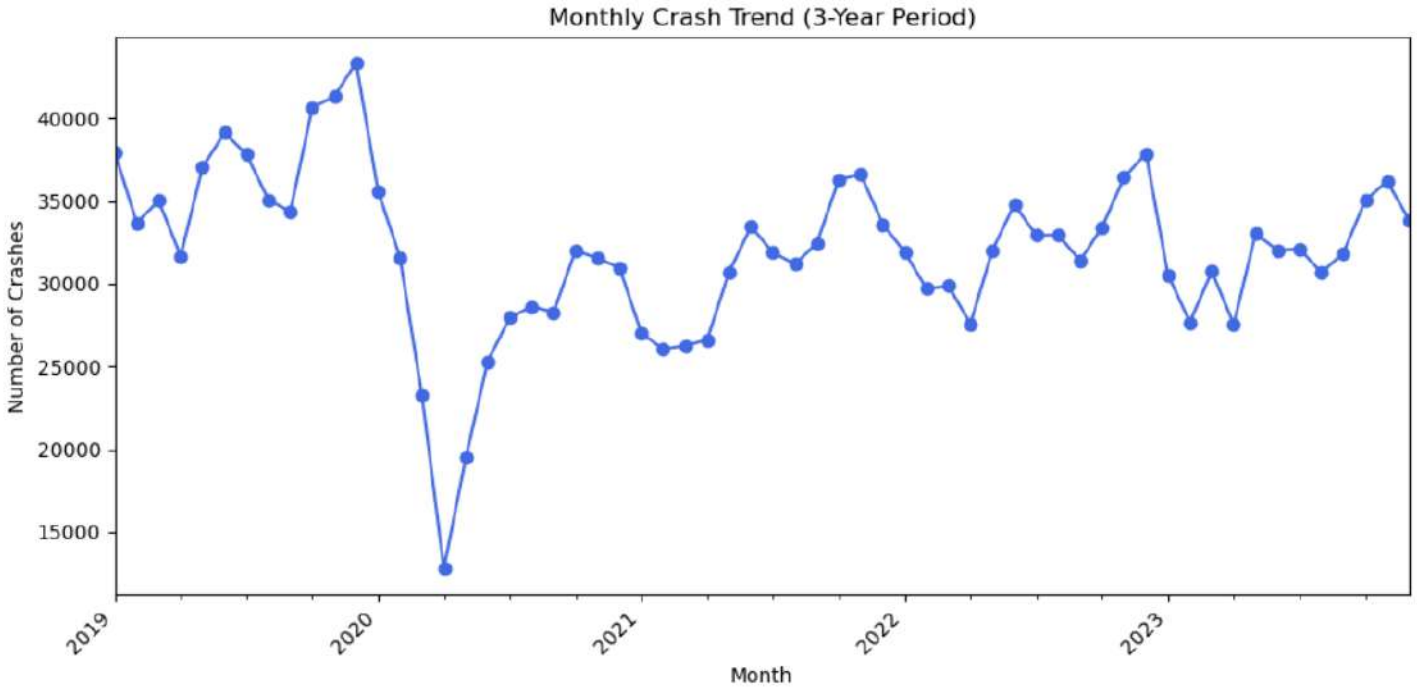

Crashes per Month (Last 3 Years)
Number of Crashes
40000
35000
30000
25000
20000
15000
2019-01
2019-11
2020-09
2021-07
2022-05
2023-03
Month


Top 10 Counties by Crash Count (NY Motor Vehicle Data)
Number of Crashes
200000
175000
150000
125000
100000
75000
50000
25000
0
SUFFOLK
NASSAU
QUEENS
KINGS
WESTCHESTER
ERIE
MONROE
BRONX
NEW YORK
ONONDAGA
County


Crashes by Day of Week
Number of Crashes
300000
250000
200000
150000
100000
50000
0
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
Sunday


Top 10 Weather Conditions During Crashes
1e6
Crash Count
1.2
1.0
0.8
0.6
0.4
0.2
0.0
Clear
Cloudy
Rain
Unknown
Snow
Sleet/Hail/Freezing Rain
Fog/Smog/Smoke
Other*
Weather Condition


Crashes by Lighting Conditions
1e6
1.2
1.0
0.8
0.6
0.4
0.2
0.0
Crash Count
Daylight
Dark-Road Lighted
Dark-Road Unlighted
Unknown
Dusk
Dawn
Lighting Condition


Monthly Crash Trend (3-Year Period)
40000
35000
30000
25000
20000
15000
Number of Crashes
2019
2020
2021
2022
2023
Month

3. Tensor-Flow: Learn simple data curation by creating a pickle with formatted datasets for training, development and testing in Tensor Flow and develop visualizations in tensor board.

Introduction:

TensorFlow is a powerful deep-learning framework used for building, training, and evaluating machine learning models. In real projects, datasets must be cleaned, formatted, and stored properly before training. Data curation helps convert raw data into a structured form for machine learning. Pickle files in Python allow us to easily store processed datasets in a compact format. In this experiment, we learn how to prepare a dataset and save it as three sets: training, development (validation), and testing. These curated datasets help in proper model training and unbiased evaluation. TensorFlow's tf.data module allows efficient loading of such datasets for model training. TensorBoard is a visualization tool inside TensorFlow that helps us monitor training.

Software Requirements:

Python 3.8 or above

TensorFlow 2.x

NumPy

scikit-learn

Jupyter Notebook / PyCharm / VS Code / Google Colab (any one

Procedure:

Step 1: Import Required Libraries

```
import numpy as np
import pickle
from sklearn.model_selection import train_test_split
import tensorflow as tf
import datetime
```

Step 2: Create or Load Dataset

```
import numpy as np
X = np.random.rand(1000, 28, 28).astype('float32')   # 1000 images of 28x28
y = np.random.randint(0, 10, size=(1000,))          # 1000 labels (0–9)
print("Shape of X (images):", X.shape)
print("Shape of y (labels):", y.shape)
```

```
print("\nSample image data (first image):")
print(X[0])
print("\nSample label:", y[0])
```

Step 3: Preprocessing (Normalization)

```
X = X / np.max(X)
```

Step 4: Split into Train, Dev, Test

```
X_rem, X_test, y_rem, y_test = train_test_split(X, y, test_size=0.15, random_state=42)
X_train, X_dev, y_train, y_dev = train_test_split(X_rem, y_rem, test_size=0.176, random_state=42)
print("Train:", X_train.shape)
print("Dev:", X_dev.shape)
print("Test:", X_test.shape)
```

Step 5: Save Datasets as Pickle Files

```
with open("train.pkl", "wb") as f:
pickle.dump((X_train, y_train), f)
with open("dev.pkl", "wb") as f:
pickle.dump((X_dev, y_dev), f)
with open("test.pkl", "wb") as f:
pickle.dump((X_test, y_test), f)
print("Pickle files saved successfully.")
```

Step 6: Load Pickle Files Using TensorFlow

```
def load_pickle(path):
with open(path, "rb") as f:
   X, y = pickle.load(f)
return tf.data.Dataset.from_tensor_slices((X, y)).batch(32)
train_ds = load_pickle("train.pkl")
dev_ds = load_pickle("dev.pkl")
test_ds = load_pickle("test.pkl")
```

Step 7: Build a Simple Neural Network Model

```
model = tf.keras.Sequential([
tf.keras.layers.Input(shape=(28, 28)),
tf.keras.layers.Reshape((28, 28, 1)),
tf.keras.layers.Conv2D(32, 3, activation="relu"),
tf.keras.layers.MaxPooling2D(),
```

```
tf.keras.layers.Flatten(),
tf.keras.layers.Dense(128, activation="relu"),
tf.keras.layers.Dense(10, activation="softmax")
])
model.compile(optimizer="adam",
        loss="sparse_categorical_crossentropy",
           metrics=["accuracy"])
```

Step 8: Create TensorBoard Log Directory

```
log_dir = "logs/run_" + datetime.datetime.now().strftime("%Y%m%d-%H%M%S")
tensorboard_callback = tf.keras.callbacks.TensorBoard(log_dir=log_dir, histogram_freq=1)
```

Step 9: Train Model with TensorBoard

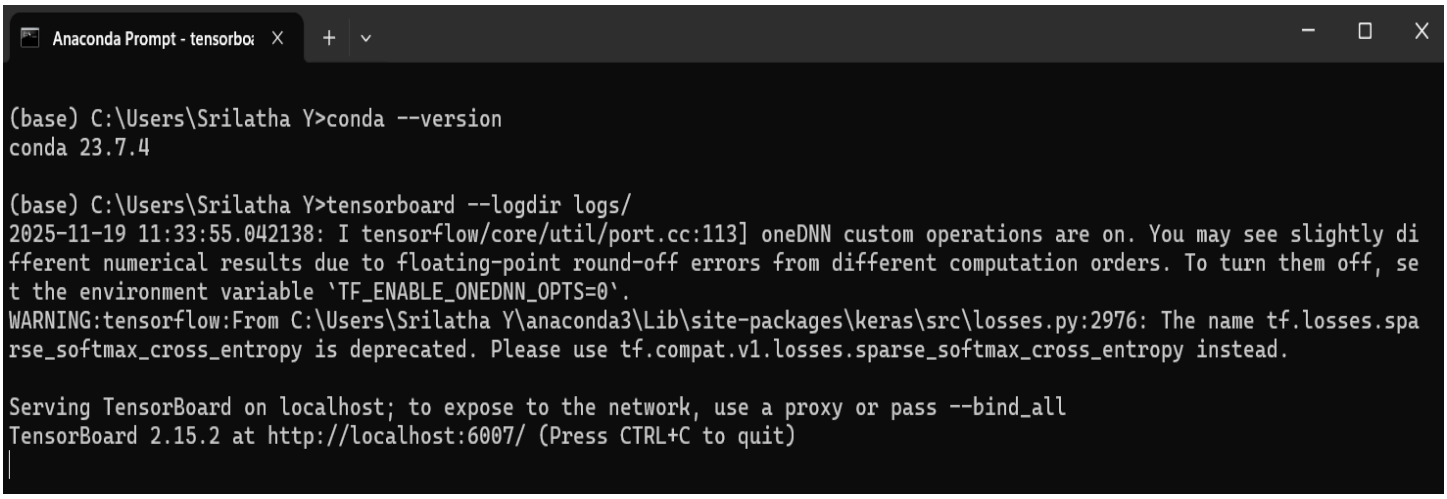
```
model.fit(train_ds,
    validation_data=dev_ds,
    epochs=5,
       callbacks=[tensorboard_callback])
```

Step 10: Open TensorBoard and View Results

Open **Anaconda Prompt / Command Prompt** and type:

```
tensorboard --logdir logs/
```

OUTPUT:



```
Shape of X (images): (1000, 28, 28)
Shape of y (labels): (1000,)

Sample image data (first image):
[[5.71788669e-01 1.16326071e-01 3.20334852e-01 3.08197767e-01
  2.19079375e-01 2.62208134e-01 8.24583590e-01 6.88976407e-01
  6.01598956e-02 2.41648912e-01 1.64396614e-01 8.24091554e-01
  7.66039610e-01 7.04157650e-01 7.67349720e-01 9.63128328e-01
  9.33474958e-01 7.04032719e-01 1.33775204e-01 5.56438267e-01
  5.53387664e-02 5.68474054e-01 5.40186644e-01 8.08603168e-02
  7.72673011e-01 2.00195506e-01 8.66302311e-01 9.90321577e-01]
 [7.19105005e-01 2.54529476e-01 7.06103593e-02 2.47178823e-01
  3.29773903e-01 7.68694934e-03 8.39405537e-01 2.87605494e-01
  6.92104340e-01 9.50798988e-01 1.38832182e-01 6.92810655e-01
  5.68645000e-01 6.98592961e-01 1.02776214e-01 1.36380821e-01
  1.26638621e-01 9.59596969e-03 5.82746744e-01 6.80507660e-01
  7.86097348e-01 7.87290335e-01 8.29945147e-01 5.04120290e-01
  7.78554559e-01 3.18779290e-01 3.80474150e-01 7.44435668e-01]
 [7.96138704e-01 4.91531432e-01 4.44807321e-01 2.59591281e-01
```

In [5]:
```
# Step 4: Split into Train (70%), Dev (15%), Test (15%)
X_rem, X_test, y_rem, y_test = train_test_split(X, y, test_size=0.15, random_state=42)
X_train, X_dev, y_train, y_dev = train_test_split(X_rem, y_rem, test_size=0.176, random_state=42)

print("Train:", X_train.shape)
print("Dev:", X_dev.shape)
print("Test:", X_test.shape)
```

```
Train: (700, 28, 28)
Dev: (150, 28, 28)
Test: (150, 28, 28)
```

In [6]:
```
# Step 5: Save Pickle Files
with open("train.pkl", "wb") as f:
    pickle.dump((X_train, y_train), f)

with open("dev.pkl", "wb") as f:
    pickle.dump((X_dev, y_dev), f)

with open("test.pkl", "wb") as f:
    pickle.dump((X_test, y_test), f)

print("Pickle files saved successfully.")
```

```
Pickle files saved successfully.
```

In [10]:
```
# Step 9: Train with TensorBoard Callback
model.fit(train_ds,
          validation_data=dev_ds,
          epochs=5,
          callbacks=[tensorboard_callback])
```

```
Epoch 1/5
22/22 [==============================] - 1s 19ms/step - loss: 2.3543 - accuracy: 0.0957 - val_loss: 2.3340 - val_accuracy: 0.04
67
Epoch 2/5
22/22 [==============================] - 0s 8ms/step - loss: 2.2906 - accuracy: 0.1243 - val_loss: 2.3044 - val_accuracy: 0.106
7
Epoch 3/5
22/22 [==============================] - 0s 9ms/step - loss: 2.2506 - accuracy: 0.1914 - val_loss: 2.3067 - val_accuracy: 0.093
3
Epoch 4/5
22/22 [==============================] - 0s 8ms/step - loss: 2.1831 - accuracy: 0.3214 - val_loss: 2.3120 - val_accuracy: 0.073
3
Epoch 5/5
22/22 [==============================] - 0s 8ms/step - loss: 2.0932 - accuracy: 0.4514 - val_loss: 2.3179 - val_accuracy: 0.080
0
```

Out[10]:
```
<keras.src.callbacks.History at 0x180a2d31c50>
```

4. Create convolutional neural networks in Tensor Flow.

Introduction:

TensorFlow is an open-source deep learning framework developed by Google for building and training machine learning models. It provides an easy-to-use high-level API, Keras, which simplifies creating neural networks. TensorFlow supports large-scale machine learning across CPUs, GPUs, and TPUs for faster computation. It uses dataflow graphs to represent mathematical operations, making it efficient for numerical computation. TensorFlow is widely used for image processing, natural language processing, and predictive analytics in both research and industry.

Software Requirements

Operating System: Windows 10/11, Linux, or macOS

Python Version: Python 3.8 – 3.11

TensorFlow Version: TensorFlow 2.x

Hardware:

- CPU (minimum)
- GPU optional (NVIDIA GPU with CUDA & cuDNN for faster training)

Required Libraries: NumPy, Matplotlib, Scikit-learn (optional), Keras (included in TensorFlow)

Tools: Jupyter Notebook / VS Code / PyCharm / Google Colab

Procedure:

Step 1: Import Required Libraries

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report, confusion_matrix
```

Step 2: Load and Normalize Dataset

```
(x_train, y_train), (x_test, y_test) = keras.datasets.cifar10.load_data()
x_train = x_train.astype("float32") / 255.0    # Normalize
x_test  = x_test.astype("float32")  / 255.0
y_train = y_train.squeeze()
y_test  = y_test.squeeze()
```

Step 3: Create Data Pipeline (Shuffle, Batch, Prefetch)

```
BATCH = 64
AUTOTUNE = tf.data.AUTOTUNE
train_ds = (tf.data.Dataset.from_tensor_slices((x_train, y_train))
        .shuffle(10000)
        .batch(BATCH)
        .prefetch(AUTOTUNE))
test_ds = (tf.data.Dataset.from_tensor_slices((x_test, y_test))
        .batch(BATCH)
        .prefetch(AUTOTUNE))
```

Step 4: Data Augmentation Layer

```
data_augmentation = keras.Sequential([
   layers.RandomFlip("horizontal"),
   layers.RandomRotation(0.07),
   layers.RandomZoom(0.07),
])
```

Step 5: Build CNN Model

```
def create_simple_cnn(input_shape=(32,32,3), num_classes=10):
inputs = keras.Input(shape=input_shape)
x = data_augmentation(inputs)    # remove if not using augmentation
x = layers.Conv2D(32, 3, padding="same", activation="relu")(x)
x = layers.BatchNormalization()(x)
x = layers.Conv2D(32, 3, padding="same", activation="relu")(x)
x = layers.MaxPooling2D()(x)
x = layers.Dropout(0.25)(x)
x = layers.Conv2D(64, 3, padding="same", activation="relu")(x)
x = layers.BatchNormalization()(x)
x = layers.MaxPooling2D()(x)
x = layers.Dropout(0.25)(x)
x = layers.Flatten()(x)
x = layers.Dense(128, activation="relu")(x)
x = layers.BatchNormalization()(x)
x = layers.Dropout(0.4)(x)
```

```
    outputs = layers.Dense(num_classes, activation="softmax")(x)
    return keras.Model(inputs, outputs)
    model = create_simple_cnn()
    model.summary()
```

Step 6: Compile the Model

```
    model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss=keras.losses.SparseCategoricalCrossentropy(),
    metrics=["accuracy"]
    )
    Step 7: Set Callbacks
    callbacks = [
    keras.callbacks.ModelCheckpoint("best_cnn.h5", save_best_only=True, monitor="val_accuracy"),
    keras.callbacks.EarlyStopping(monitor="val_loss", patience=6, restore_best_weights=True)
    ]
```

Step 8: Train the Model

```
history = model.fit(
    train_ds,
    epochs=25,
    validation_data=test_ds,
    callbacks=callbacks
    )
```

Step 9: Evaluate on Test Data

```
    test_loss, test_acc = model.evaluate(test_ds)
    print("Test Accuracy:", test_acc)
    print("Test Loss:", test_loss)
```

Step 10: Save Final Model

```
    model.save("cnn_saved_model")
```

Step 11: Plot Accuracy & Loss Graphs

```
    plt.plot(history.history["accuracy"], label="'Train Accuracy")
    plt.plot(history.history["val_accuracy"], label="Val Accuracy")
    plt.legend()
    plt.show()
```

```
plt.plot(history.history["loss"], label="Train Loss")
plt.plot(history.history["val_loss"], label="Val Loss")
plt.legend()
plt.show()
```

Step 12: Classification Report & Confusion Matrix

```
y_pred_probs = model.predict(x_test)
y_pred = np.argmax(y_pred_probs, axis=1)
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

OUTPUT:

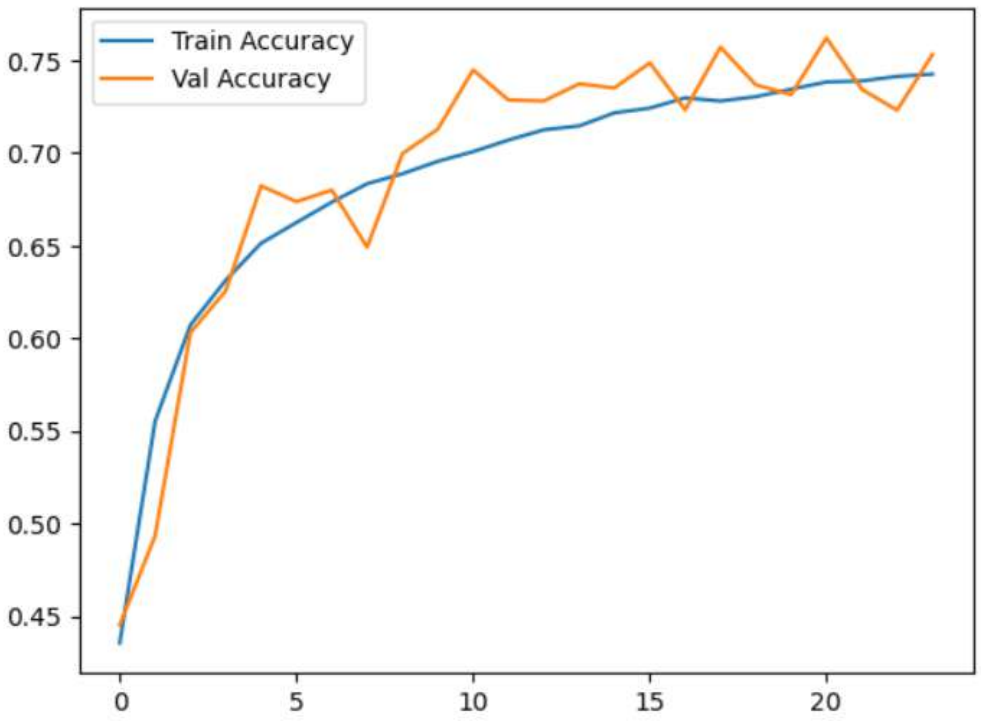
```
In [5]: # Load and Normalize Dataset
        (x_train, y_train), (x_test, y_test) = keras.datasets.cifar10.load_data()

        x_train = x_train.astype("float32") / 255.0    # Normalize
        x_test  = x_test.astype("float32")  / 255.0

        y_train = y_train.squeeze()
        y_test  = y_test.squeeze()

        Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
        170498071/170498071 [==============================] - 28s 0us/step
```

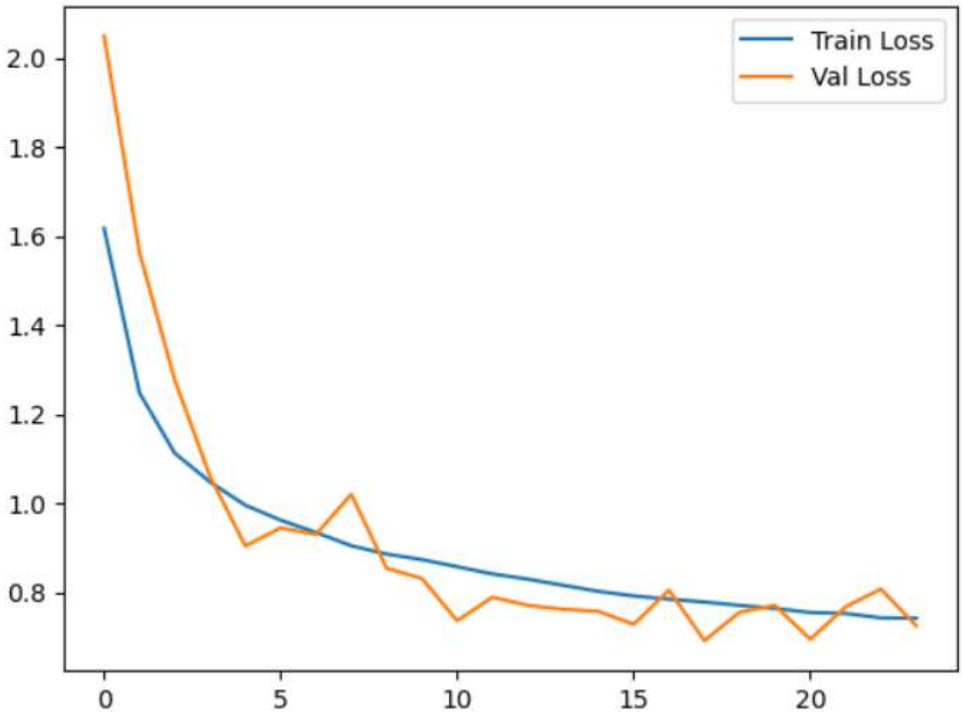
```
Model: "model"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, 32, 32, 3)]       0

 sequential (Sequential)     (None, 32, 32, 3)         0

 conv2d (Conv2D)             (None, 32, 32, 32)        896

 batch_normalization (Batch  (None, 32, 32, 32)        128
 Normalization)

 conv2d_1 (Conv2D)           (None, 32, 32, 32)        9248

 max_pooling2d (MaxPooling2  (None, 16, 16, 32)        0
 D)

 dropout (Dropout)           (None, 16, 16, 32)        0

 conv2d_2 (Conv2D)           (None, 16, 16, 64)        18496

 batch_normalization_1 (Bat  (None, 16, 16, 64)        256
 chNormalization)

 max_pooling2d_1 (MaxPoolin  (None, 8, 8, 64)          0
 g2D)

 dropout_1 (Dropout)         (None, 8, 8, 64)          0

 flatten (Flatten)           (None, 4096)              0

  dense (Dense)              (None, 128)               524416

  batch_normalization_2 (Bat  (None, 128)               512
  chNormalization)

  dropout_2 (Dropout)        (None, 128)               0

  dense_1 (Dense)            (None, 10)                1290

 =================================================================
 Total params: 555242 (2.12 MB)
 Trainable params: 554794 (2.12 MB)
 Non-trainable params: 448 (1.75 KB)
 _________________________________________________________________
```

```
Epoch 1/25
782/782 [==============================] - 28s 34ms/step - loss: 1.6168 - accuracy: 0.4356 - val_loss: 2.0488 - val_accuracy:
0.4453
Epoch 2/25
782/782 [==============================] - 26s 34ms/step - loss: 1.2481 - accuracy: 0.5552 - val_loss: 1.5634 - val_accuracy:
0.4934
Epoch 3/25
782/782 [==============================] - 27s 34ms/step - loss: 1.1124 - accuracy: 0.6071 - val_loss: 1.2766 - val_accuracy:
0.6030
Epoch 4/25
782/782 [==============================] - 27s 34ms/step - loss: 1.0478 - accuracy: 0.6312 - val_loss: 1.0602 - val_accuracy:
0.6255
Epoch 5/25
782/782 [==============================] - 27s 34ms/step - loss: 0.9956 - accuracy: 0.6512 - val_loss: 0.9042 - val_accuracy:
0.6821
Epoch 6/25
782/782 [==============================] - 27s 35ms/step - loss: 0.9621 - accuracy: 0.6623 - val_loss: 0.9446 - val_accuracy:
0.6736
Epoch 7/25
782/782 [==============================] - 27s 35ms/step - loss: 0.9345 - accuracy: 0.6733 - val_loss: 0.9300 - val_accuracy:
0.6798
Epoch 8/25
782/782 [==============================] - 27s 35ms/step - loss: 0.9046 - accuracy: 0.6832 - val_loss: 1.0201 - val_accuracy:
0.6490
Epoch 9/25
782/782 [==============================] - 27s 34ms/step - loss: 0.8861 - accuracy: 0.6886 - val_loss: 0.8542 - val_accuracy:
```

In [12]:
```
# Evaluate on Test Data
test_loss, test_acc = model.evaluate(test_ds)
print("Test Accuracy:", test_acc)
print("Test Loss:", test_loss)
```

```
157/157 [==============================] - 2s 11ms/step - loss: 0.6917 - accuracy: 0.7569
Test Accuracy: 0.7569000124931335
Test Loss: 0.6916780471801758
```






```
313/313 [==============================] - 2s 6ms/step
              precision    recall  f1-score   support

           0       0.82      0.80      0.81      1000
           1       0.87      0.91      0.89      1000
           2       0.81      0.55      0.66      1000
           3       0.70      0.44      0.54      1000
           4       0.66      0.74      0.70      1000
           5       0.75      0.61      0.67      1000
           6       0.61      0.92      0.74      1000
           7       0.77      0.85      0.81      1000
           8       0.88      0.87      0.87      1000
           9       0.78      0.88      0.83      1000

    accuracy                           0.76     10000
   macro avg       0.76      0.76      0.75     10000
weighted avg       0.76      0.76      0.75     10000

[[798  23  21   9  17   2  19   9  43  59]
 [  4 911   1   1   1   0   3   1   8  70]
 [ 52   6 551  24 103  37 144  48  16  19]
 [ 19  14  35 438  83 137 167  51  19  37]
 [ 15   2  26  21 739   8 124  55   7   3]
 [  9   4  21  86  82 608  88  73  10  19]
 [  5   5   7  19  21   6 923   3   5   6]
 [ 10   3  12  18  61  13  18 850   2  13]
 [ 46  24   4   2   8   1  11   5 869  30]
 [ 18  58   1   6   4   0   5  12  14 882]]
```

5. Image recognition (or image classification) : identifying images and categorizing them in one of several predefined distinct classes using neural network models.

Introduction:

Image recognition (image classification) is the task of assigning an input image to one of a fixed set of categories. Modern image classification uses convolutional neural networks (CNNs) which learn hierarchical visual features (edges → textures → parts → objects). Training requires labeled images for each class, preprocessing (resize, normalize), a model architecture, and an optimization loop. After training, the model generalizes to classify unseen images. Evaluation uses metrics such as accuracy and confusion matrices to understand per-class performance.

Software / hardware requirements

Python 3.8+

TensorFlow 2.10+ (includes Keras API) — pip install tensorflow

numpy, matplotlib, scikit-learn, pillow — pip install numpy matplotlib scikit-learn pillow

Procedure:

Step 1: Install & Import Libraries

```
!pip install --quiet tensorflow matplotlib scikit-learn pillow
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.metrics import classification_report, confusion_matrix
print("TensorFlow version:", tf.__version__)
```

Step 2: Load the CIFAR-10 Dataset

```
(x_train, y_train), (x_test, y_test) = keras.datasets.cifar10.load_data()
y_train = y_train.ravel()
y_test = y_test.ravel()
class_names = ["airplane","automobile","bird","cat","deer","dog","frog","horse","ship","truck"]
print("Training data:", x_train.shape)
print("Testing data:", x_test.shape)
print("Classes:", class_names)
```

Step 3: Display Sample Images

```
plt.figure(figsize=(10,4))
for i in range(10):
plt.subplot(2,5,i+1)
plt.imshow(x_train[i])
plt.title(class_names[y_train[i]])
plt.axis('off')
plt.show()
```

Step 4: Preprocess & Prepare tf.data Datasets

```
IMG_SIZE = (32, 32)
BATCH_SIZE = 64
AUTOTUNE = tf.data.AUTOTUNE
def preprocess(images, labels):
images = tf.cast(images, tf.float32) / 255.0
return images, labels
train_ds = tf.data.Dataset.from_tensor_slices((x_train, y_train))
train_ds = train_ds.shuffle(10000).map(preprocess).batch(BATCH_SIZE).prefetch(AUTOTUNE)
test_ds = tf.data.Dataset.from_tensor_slices((x_test, y_test))
test_ds = test_ds.map(preprocess).batch(BATCH_SIZE).prefetch(AUTOTUNE)
```

Step 5: Build the CNN Model

```
def make_cnn(input_shape=(32,32,3), num_classes=10):
inputs = keras.Input(shape=input_shape)
x = layers.Conv2D(32, (3,3), padding='same', activation='relu')(inputs)
x = layers.MaxPooling2D()(x)
x = layers.Conv2D(64, (3,3), padding='same', activation='relu')(x)
x = layers.MaxPooling2D()(x)
x = layers.Conv2D(128, (3,3), padding='same', activation='relu')(x)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(128, activation='relu')(x)
outputs = layers.Dense(num_classes, activation='softmax')(x)
return keras.Model(inputs, outputs)
model = make_cnn()
model.summary()
```

Step 6: Compile the Model

```
model.compile(
  optimizer='adam',
  loss='sparse_categorical_crossentropy',
  metrics=['accuracy']
)
```

Step 7: Train the Model

```
history = model.fit(
  train_ds,
  validation_data=test_ds,
  epochs=10
)
```

Step 8: Plot Accuracy & Loss Curves

```
plt.figure(figsize=(12,4))
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label="Train Acc")
plt.plot(history.history['val_accuracy'], label="Test Acc")
plt.title("Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label="Train Loss")
plt.plot(history.history['val_loss'], label="Test Loss")
plt.title("Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()
```

Step 9: Evaluate the Model

```
test_loss, test_acc = model.evaluate(test_ds)
print("Test Accuracy:", test_acc)
print("Test Loss:", test_loss)
```

Step 10: Classification Report & Confusion Matrix

```
y_true = []
y_pred = []
for images, labels in test_ds:
    preds = model.predict(images)
    y_pred.extend(np.argmax(preds, axis=1))
    y_true.extend(labels.numpy())
print(classification_report(y_true, y_pred, target_names=class_names))
```

Step 11: Single Image Prediction

```
import random
idx = random.randint(0, x_test.shape[0]-1)
img = x_test[idx]
img_norm = img.astype('float32') / 255.0
img_norm = np.expand_dims(img_norm, axis=0)
pred = model.predict(img_norm)[0]
pred_label = class_names[np.argmax(pred)]
confidence = np.max(pred)
print("True label:", class_names[y_test[idx]])
print("Predicted:", pred_label)
print("Confidence:", confidence)
plt.imshow(img)
plt.axis('off')
plt.show()
```

OUTPUT:

In [3]:
```
# Importing libraries and installing required packages

!pip install --quiet tensorflow matplotlib scikit-learn pillow

import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.metrics import classification_report, confusion_matrix

print("TensorFlow version:", tf.__version__)
```

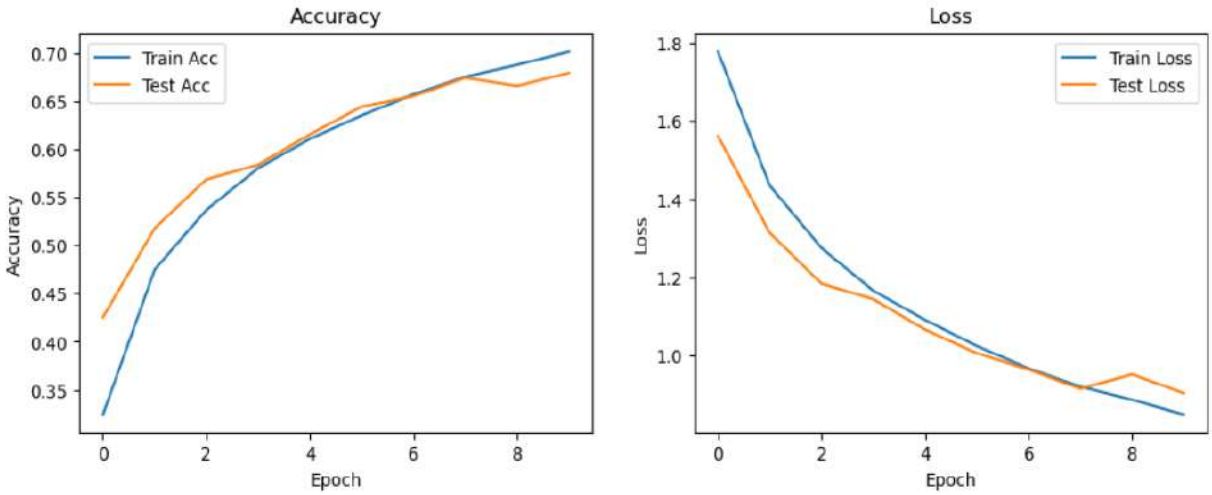
```
TensorFlow version: 2.15.0
```

In [4]:
```
# Loading CIFAR-10 dataset

(x_train, y_train), (x_test, y_test) = keras.datasets.cifar10.load_data()

y_train = y_train.ravel()
y_test = y_test.ravel()

class_names = ["airplane","automobile","bird","cat","deer","dog","frog","horse","ship","truck"]

print("Training data:", x_train.shape)
print("Testing data:", x_test.shape)
print("Classes:", class_names)
```

```
Training data: (50000, 32, 32, 3)
Testing data: (10000, 32, 32, 3)
Classes: ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
```

```
Model: "model"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, 32, 32, 3)]       0

 conv2d (Conv2D)             (None, 32, 32, 32)        896

 max_pooling2d (MaxPooling2  (None, 16, 16, 32)        0
 D)

 conv2d_1 (Conv2D)           (None, 16, 16, 64)        18496

 max_pooling2d_1 (MaxPoolin  (None, 8, 8, 64)          0
 g2D)

 conv2d_2 (Conv2D)           (None, 8, 8, 128)         73856

 global_average_pooling2d (  (None, 128)               0
 GlobalAveragePooling2D)

 dense (Dense)               (None, 128)               16512

 dense_1 (Dense)             (None, 10)                1290

=================================================================
Total params: 111050 (433.79 KB)
Trainable params: 111050 (433.79 KB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

```
782/782 [==============================] - 13s 15ms/step - loss: 1.7778 - accuracy: 0.3243 - val_loss: 1.5619 - val_accuracy:
0.4253
Epoch 2/10
782/782 [==============================] - 12s 15ms/step - loss: 1.4352 - accuracy: 0.4748 - val_loss: 1.3139 - val_accuracy:
0.5177
Epoch 3/10
782/782 [==============================] - 12s 15ms/step - loss: 1.2753 - accuracy: 0.5368 - val_loss: 1.1844 - val_accuracy:
0.5685
Epoch 4/10
782/782 [==============================] - 12s 16ms/step - loss: 1.1664 - accuracy: 0.5801 - val_loss: 1.1445 - val_accuracy:
0.5837
Epoch 5/10
782/782 [==============================] - 12s 15ms/step - loss: 1.0914 - accuracy: 0.6105 - val_loss: 1.0666 - val_accuracy:
0.6152
Epoch 6/10
782/782 [==============================] - 12s 16ms/step - loss: 1.0248 - accuracy: 0.6349 - val_loss: 1.0064 - val_accuracy:
0.6440
Epoch 7/10
782/782 [==============================] - 13s 16ms/step - loss: 0.9673 - accuracy: 0.6567 - val_loss: 0.9647 - val_accuracy:
0.6548
Epoch 8/10
782/782 [==============================] - 12s 16ms/step - loss: 0.9210 - accuracy: 0.6749 - val_loss: 0.9154 - val_accuracy:
0.6739
Epoch 9/10
782/782 [==============================] - 13s 16ms/step - loss: 0.8873 - accuracy: 0.6873 - val_loss: 0.9523 - val_accuracy:
0.6654
Epoch 10/10
782/782 [==============================] - 13s 16ms/step - loss: 0.8492 - accuracy: 0.7011 - val_loss: 0.9039 - val_accuracy:
0.6790
```




```
2/2 [==============================] - 0s 16ms/step
2/2 [==============================] - 0s 15ms/step
1/1 [==============================] - 0s 45ms/step
```

	precision	recall	f1-score	support
airplane	0.74	0.73	0.73	1000
automobile	0.73	0.89	0.80	1000
bird	0.53	0.66	0.59	1000
cat	0.51	0.39	0.44	1000
deer	0.63	0.67	0.65	1000
dog	0.48	0.75	0.58	1000
frog	0.85	0.67	0.74	1000
horse	0.87	0.59	0.70	1000
ship	0.91	0.71	0.80	1000
truck	0.82	0.74	0.78	1000
accuracy			0.68	10000
macro avg	0.71	0.68	0.68	10000
weighted avg	0.71	0.68	0.68	10000

```
1/1 [==============================] - 0s 23ms/step
True label: frog
Predicted: frog
Confidence: 0.41438437
```

6. OpenCV: Develop an online writing Whiteboard with minimal features for online classes

Introduction

OpenCV (Open Source Computer Vision Library) is a powerful open-source toolkit used for real-time computer vision and image processing. It provides hundreds of functions to handle tasks like image filtering, object detection, face recognition, video analysis, and more. A minimal online writing whiteboard built with OpenCV provides a lightweight drawing surface for live teaching and quick demos. This implementation creates a resizable blank canvas where the teacher (or student) can draw with the mouse, switch colors, erase, clear the board, and save snapshots.

Software requirements

Python 3.8+

Packages: opencv-python, numpy

Install with: pip install opencv-python numpy

Procedure:

Step 1: Enable GUI backend for OpenCV inside Jupyter

```
%gui tk
```

Step 2: Import libraries and prepare whiteboard

```
import cv2
import numpy as np
from datetime import datetime
CANVAS_HEIGHT = 600
CANVAS_WIDTH = 1000
BACKGROUND_COLOR = (255, 255, 255)
canvas = np.full((CANVAS_HEIGHT, CANVAS_WIDTH, 3), BACKGROUND_COLOR, dtype=np.uint8)
drawing = False
ix, iy = -1, -1
color = (0, 0, 0)
thickness = 4
eraser = False
```

Step 3: Mouse callback

```
def mouse_events(event, x, y, flags, param):
    global drawing, ix, iy, color, thickness, eraser, canvas
```

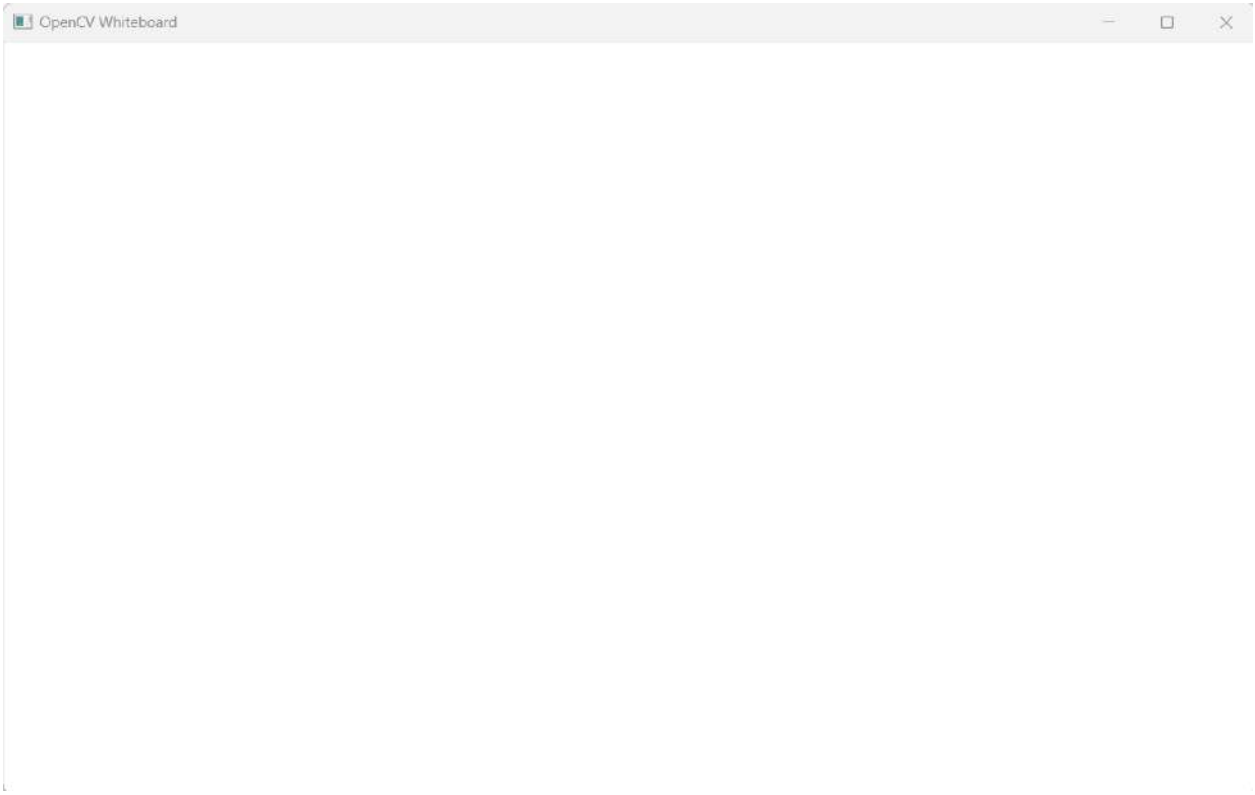
```
    if event == cv2.EVENT_LBUTTONDOWN:
        drawing = True
        ix, iy = x, y
        draw_color = BACKGROUND_COLOR if eraser else color
        cv2.circle(canvas, (x, y), thickness // 2, draw_color, -1)
    elif event == cv2.EVENT_MOUSEMOVE and drawing:
        draw_color = BACKGROUND_COLOR if eraser else color
        cv2.line(canvas, (ix, iy), (x, y), draw_color, thickness)
        ix, iy = x, y
    elif event == cv2.EVENT_LBUTTONUP:
            drawing = False
```

Step 4: Main Whiteboard Loop

```
    window_name = "OpenCV Whiteboard"
    cv2.namedWindow(window_name)
    cv2.setMouseCallback(window_name, mouse_events)
    print("Controls:")
    print("1=Black 2=Red 3=Green 4=Blue e=Eraser c=Clear s=Save q=Quit")
while True:
    cv2.imshow(window_name, canvas)
    key = cv2.waitKey(10) & 0xFF
    if key == ord('1'): color = (0, 0, 0); eraser = False
    elif key == ord('2'): color = (0, 0, 255); eraser = False
    elif key == ord('3'): color = (0, 255, 0); eraser = False
    elif key == ord('4'): color = (255, 0, 0); eraser = False
    elif key == ord('e'): eraser = not eraser
    elif key == ord('['): thickness = max(1, thickness - 1)
    elif key == ord(']'): thickness += 1
    elif key == ord('c'): canvas[:] = BACKGROUND_COLOR
    elif key == ord('s'):
        filename = f"whiteboard_{datetime.now().strftime('%Y%m%d_%H%M%S')}.png"
        cv2.imwrite(filename, canvas)
        print("Saved:", filename)
    elif key == ord('q') or key == 27:
```

```
    break
cv2.destroyAllWindows()
```

OUTPUT:



7. Keras: Recognize handwritten digits from MNIST using Keras

Introduction:

Keras is a high-level deep learning library that makes building neural networks simple and fast. It provides an easy, user-friendly API to create models for tasks like image classification, NLP, and regression. The MNIST (Modified National Institute of Standards and Technology) dataset is a classic benchmark of 70,000 grayscale images of handwritten digits (0–9). We'll use TensorFlow's Keras API to build a simple Convolutional Neural Network (CNN) that classifies these digits. The notebook will load data, preprocess it, define and train a model, evaluate performance, and show example predictions.

Software Requirements:

Python 3.8+ recommended.

Jupyter Notebook or JupyterLab.

TensorFlow 2.x (which includes Keras).

Procedure:

Step 1: Imports & environment check

```
import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt
import os
print("TensorFlow version:", tf.__version__)
```

Step 2: Load MNIST dataset

```
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
print("Train images:", x_train.shape, "Train labels:", y_train.shape)
print("Test images:", x_test.shape, "Test labels:", y_test.shape)
```

Step 3: Quick data exploration (view a few samples)

```
plt.figure(figsize=(8,4))
for i in range(6):
  plt.subplot(2,3,i+1)
  plt.imshow(x_train[i], cmap='gray')
  plt.title(f"Label: {y_train[i]}")
```

```
    plt.axis('off')
plt.tight_layout()
```

Step 4: Preprocess data (reshape & normalize)

```
# Convert to float32 and normalize
x_train = x_train.astype('float32') / 255.0
x_test  = x_test.astype('float32')  / 255.0
# Add channel dimension (MNIST is grayscale so channel=1)
x_train = np.expand_dims(x_train, -1)  # shape -> (num_samples, 28, 28, 1)
x_test  = np.expand_dims(x_test, -1)
print("After preprocess - x_train shape:", x_train.shape)
```

Step 5: Prepare labels

```
print("y_train dtype:", y_train.dtype, "unique labels:", np.unique(y_train))
```

Step 6: Build the CNN model

```
model = keras.Sequential([
    keras.layers.Input(shape=(28,28,1), name="input_layer"),
    keras.layers.Conv2D(32, (3,3), activation='relu', name="conv1"),
    keras.layers.MaxPooling2D((2,2), name="pool1"),
    keras.layers.Conv2D(64, (3,3), activation='relu', name="conv2"),
    keras.layers.MaxPooling2D((2,2), name="pool2"),
    keras.layers.Flatten(name="flatten"),
    keras.layers.Dense(128, activation='relu', name="dense1"),
    keras.layers.Dropout(0.4, name="dropout"),
    keras.layers.Dense(10, activation='softmax', name="output")  # 10 classes
])
model.summary()
```

Step 7: Compile the model

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',  # use sparse because labels are integers
    metrics=['accuracy']
)
```

Step 8: Train the model

```
EPOCHS = 6
```

```
BATCH_SIZE = 128
history = model.fit(
   x_train, y_train,
   validation_split=0.1,  # keep 10% of train for validation
   epochs=EPOCHS,
   batch_size=BATCH_SIZE,
   verbose=2
)
```

Step 9: Plot training history (loss & accuracy)

```
plt.figure(figsize=(12,4))
plt.subplot(1,2,1)
plt.plot(history.history['loss'], label='train_loss')
plt.plot(history.history['val_loss'], label='val_loss')
plt.title('Loss')
plt.xlabel('Epoch')
plt.legend()
plt.subplot(1,2,2)
plt.plot(history.history['accuracy'], label='train_acc')
plt.plot(history.history['val_accuracy'], label='val_acc')
plt.title('Accuracy')
plt.xlabel('Epoch')
plt.legend()
plt.tight_layout()
```

Step 10: Evaluate on the test set

```
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print(f"Test loss: {test_loss:.4f}  Test accuracy: {test_acc:.4f}")
```

Step 11: Make predictions and show examples

```
pred_probs = model.predict(x_test)      # probabilities for each class
pred_labels = np.argmax(pred_probs, axis=1)
plt.figure(figsize=(9,6))
num = 12
for i in range(num):
   plt.subplot(3,4,i+1)
```

```
    plt.imshow(x_test[i].squeeze(), cmap='gray')
    plt.title(f"Pred: {pred_labels[i]}, True: {y_test[i]}")
    plt.axis('off')
plt.tight_layout()
```

Step 12: Save the model

```
save_dir = "mnist_cnn_model"
model.save(save_dir)   # creates a directory with the saved model
print("Model saved to", save_dir)
```

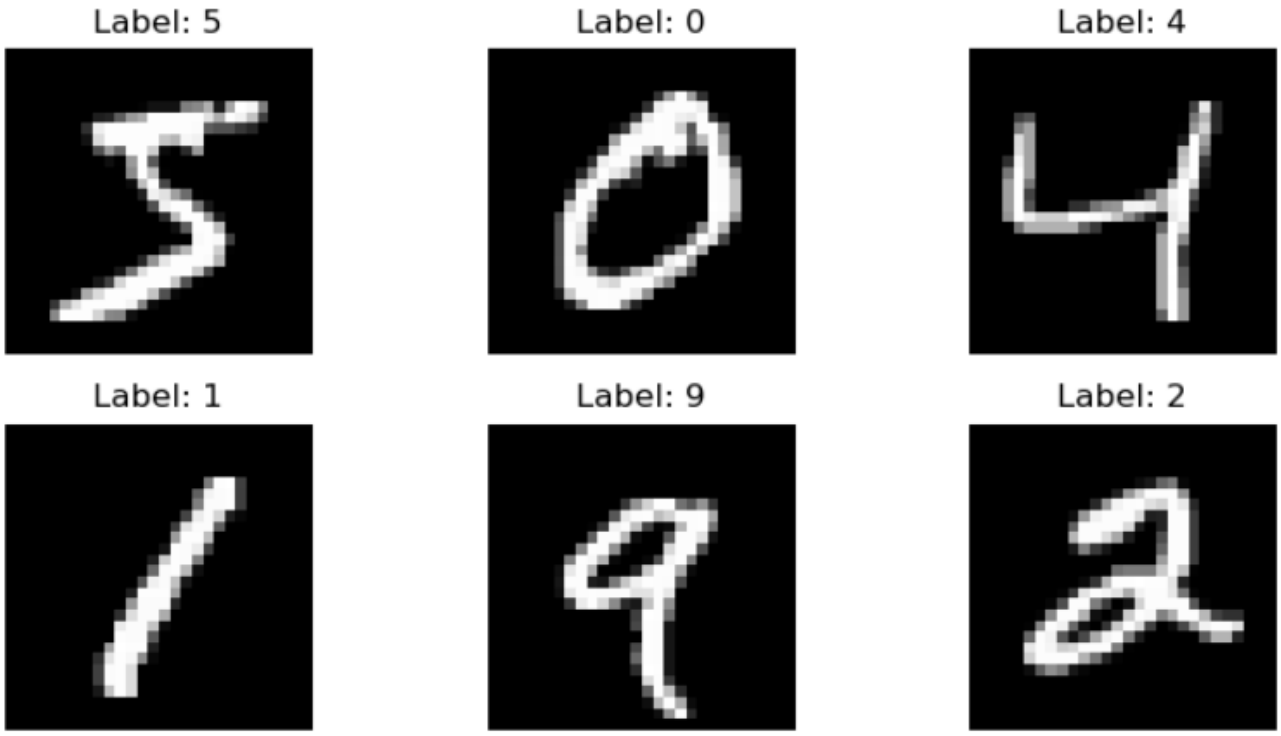
OUTPUT:

In [1]:
```
# Imports & environment check
# What this does: imports the necessary libraries and prints their versions.

import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt
import os
print("TensorFlow version:", tf.__version__)
```

```
WARNING:tensorflow:From C:\Users\Srilatha Y\anaconda3\Lib\site-packages\keras\src\losses.py:2976: The name tf.losses.sparse_sof
tmax_cross_entropy is deprecated. Please use tf.compat.v1.losses.sparse_softmax_cross_entropy instead.

TensorFlow version: 2.15.0
```

In [2]:
```
# Load MNIST dataset
# What this does: loads MNIST from Keras (train and test splits).
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
print("Train images:", x_train.shape, "Train labels:", y_train.shape)
print("Test images:", x_test.shape, "Test labels:", y_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 [==============================] - 2s 0us/step
Train images: (60000, 28, 28) Train labels: (60000,)
Test images: (10000, 28, 28) Test labels: (10000,)
```



In [4]:
```
# Preprocess data (reshape & normalize)
# What this does: scales pixel values to [0,1] and adds a channel dimension for Keras.

# Convert to float32 and normalize
x_train = x_train.astype('float32') / 255.0
x_test  = x_test.astype('float32')  / 255.0

# Add channel dimension (MNIST is grayscale so channel=1)
x_train = np.expand_dims(x_train, -1)  # shape -> (num_samples, 28, 28, 1)
x_test  = np.expand_dims(x_test, -1)

print("After preprocess - x_train shape:", x_train.shape)
```

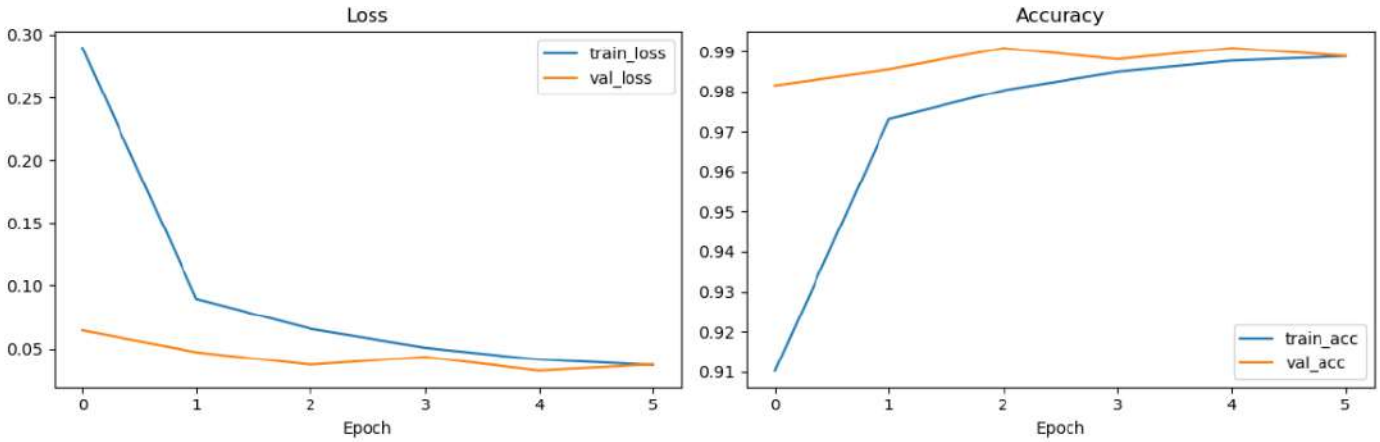
```
After preprocess - x_train shape: (60000, 28, 28, 1)
```

In [5]:
```
# Labels - using sparse integer labels (no one-hot needed)
# What this does: confirms label dtype and unique classes.
print("y_train dtype:", y_train.dtype, "unique labels:", np.unique(y_train))
```

```
y_train dtype: uint8 unique labels: [0 1 2 3 4 5 6 7 8 9]
```

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv1 (Conv2D)              (None, 26, 26, 32)        320

 pool1 (MaxPooling2D)        (None, 13, 13, 32)        0

 conv2 (Conv2D)              (None, 11, 11, 64)        18496

 pool2 (MaxPooling2D)        (None, 5, 5, 64)          0

 flatten (Flatten)           (None, 1600)              0

 dense1 (Dense)              (None, 128)               204928

 dropout (Dropout)           (None, 128)               0

 output (Dense)              (None, 10)                1290

=================================================================
Total params: 225034 (879.04 KB)
Trainable params: 225034 (879.04 KB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

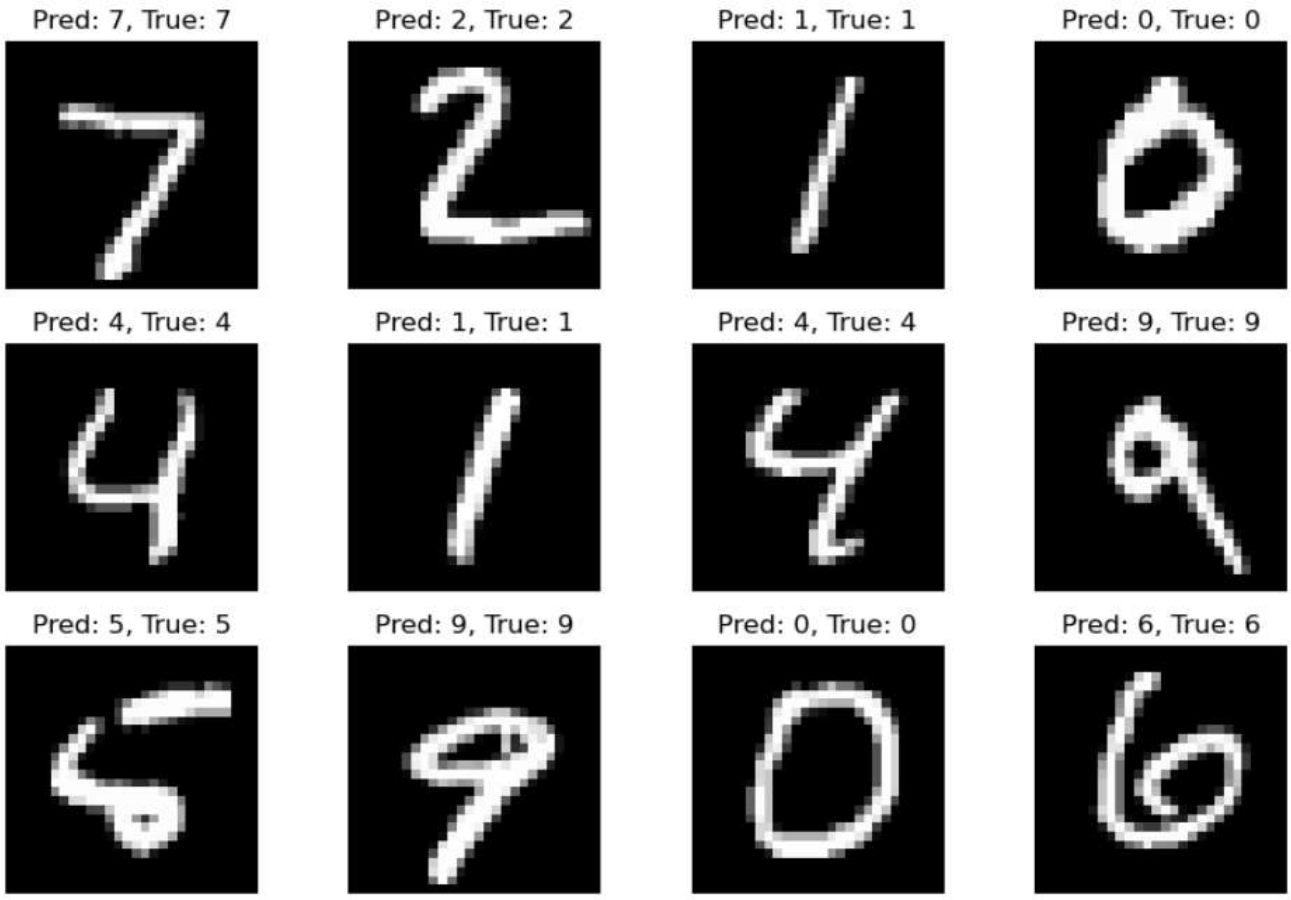
```
Epoch 1/6
WARNING:tensorflow:From C:\Users\Srilatha Y\anaconda3\Lib\site-packages\keras\src\utils\tf_utils.py:492: The name tf.ragged.RaggedTensorValue is deprecated. Please use tf.compat.v1.ragged.RaggedTensorValue instead.

WARNING:tensorflow:From C:\Users\Srilatha Y\anaconda3\Lib\site-packages\keras\src\engine\base_layer_utils.py:384: The name tf.executing_eagerly_outside_functions is deprecated. Please use tf.compat.v1.executing_eagerly_outside_functions instead.

422/422 - 31s - loss: 0.2895 - accuracy: 0.9102 - val_loss: 0.0644 - val_accuracy: 0.9815 - 31s/epoch - 74ms/step
Epoch 2/6
422/422 - 28s - loss: 0.0895 - accuracy: 0.9731 - val_loss: 0.0470 - val_accuracy: 0.9855 - 28s/epoch - 67ms/step
Epoch 3/6
422/422 - 30s - loss: 0.0655 - accuracy: 0.9803 - val_loss: 0.0369 - val_accuracy: 0.9908 - 30s/epoch - 72ms/step
Epoch 4/6
422/422 - 37s - loss: 0.0506 - accuracy: 0.9849 - val_loss: 0.0432 - val_accuracy: 0.9880 - 37s/epoch - 87ms/step
Epoch 5/6
422/422 - 32s - loss: 0.0411 - accuracy: 0.9876 - val_loss: 0.0322 - val_accuracy: 0.9908 - 32s/epoch - 77ms/step
Epoch 6/6
422/422 - 29s - loss: 0.0366 - accuracy: 0.9887 - val_loss: 0.0371 - val_accuracy: 0.9888 - 29s/epoch - 70ms/step
```



In [10]:
```
# Evaluate on the test set
# What this does: prints test loss and test accuracy.

test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print(f"Test loss: {test_loss:.4f}  Test accuracy: {test_acc:.4f}")
```

```
313/313 - 2s - loss: 0.0342 - accuracy: 0.9886 - 2s/epoch - 5ms/step
Test loss: 0.0342  Test accuracy: 0.9886
```

```
313/313 [==============================] - 2s 5ms/step
```



In [12]:
```
# Save the model
# What this does: saves the trained model to disk in TensorFlow SavedModel format.
save_dir = "mnist_cnn_model"
model.save(save_dir)   # creates a directory with the saved model
print("Model saved to", save_dir)

# To load later: loaded_model = keras.models.load_model('mnist_cnn_model')
```

```
INFO:tensorflow:Assets written to: mnist_cnn_model\assets
```

```
INFO:tensorflow:Assets written to: mnist_cnn_model\assets
```

```
Model saved to mnist_cnn_model
```

8. Scikit-learn: Write a Python program using Scikit-learn to split the iris dataset into 70% train data and 30% test data. Out of total 150 records, the training set will contain 120 records and the test set contains 30 of those records. Print both datasets.

Introduction:

Scikit-Learn is a popular Python library for machine learning that provides simple and efficient tools for data preprocessing, classification, regression, clustering, and model evaluation. Scikit-Learn is widely used for building and experimenting with machine learning models. The Iris dataset is a classic small dataset for classification. It contains 150 samples of iris flowers with four features (sepal length, sepal width, petal length, petal width) and a target (species). This example shows how to load the dataset with scikit-learn, split it into training and test sets (70/30 split), and print the resulting datasets in a readable form.

Software requirements

Python 3.8+ (3.9/3.10 are fine)

jupyterlab or notebook

scikit-learn

pandas

numpy

Procedure:

Step 1: Imports and environment check

```
import numpy as np
import pandas as pd
from sklearn import datasets
from sklearn.model_selection import train_test_split
print("Python version:", __import__("sys").version.split()[0])
print("NumPy version:", np.__version__)
print("Pandas version:", pd.__version__)
import sklearn
print("scikit-learn version:", sklearn.__version__)
```

Step 2: Load the Iris dataset

```
iris = datasets.load_iris()
X = iris.data          # features (shape 150 x 4)
```

```
y = iris.target        # targets (0,1,2)
feature_names = iris.feature_names
target_names = iris.target_names
print("Feature names:", feature_names)
print("Target names:", target_names)
print("Full dataset shape (X):", X.shape)
print("Full target shape (y):", y.shape)
```

Step 3: Create a DataFrame

```
df = pd.DataFrame(X, columns=feature_names)
df['target'] = y
df['target_name'] = df['target'].apply(lambda i: target_names[i])
df.head(8)
```

Step 4: Split the dataset into 70% train and 30% test

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
X, y,
train_size=120,   # exact number
stratify=y,
random_state=42
)
print("X_train shape:", X_train.shape)
print("X_test shape :", X_test.shape)
print("y_train length:", len(y_train))
print("y_test length :", len(y_test))
assert X_train.shape[0] == 120, "Training set should contain 120 records"
assert X_test.shape[0] == 30, "Test set should contain 30 records"
```

Step 5: Convert the splits to DataFrames

```
# Training set DataFrame
df_train = pd.DataFrame(X_train, columns=feature_names)
df_train['target'] = y_train
df_train['target_name'] = df_train['target'].apply(lambda i: target_names[i])
# Test set DataFrame
df_test = pd.DataFrame(X_test, columns=feature_names)
```

```
df_test['target'] = y_test
df_test['target_name'] = df_test['target'].apply(lambda i: target_names[i])
# Print summary and then the full DataFrames
print("=== Training set summary ===")
print(df_train.shape)
display(df_train)   # in Jupyter this shows a nice table
print("\n=== Test set summary ===")
print(df_test.shape)
display(df_test)    # in Jupyter this shows a nice table
```

Step 6: Save to CSV for inspection

```
df_train.to_csv("iris_train_120.csv", index=False)
df_test.to_csv("iris_test_30.csv", index=False)
print("Saved iris_train_120.csv and iris_test_30.csv in the current notebook directory.")
```

OUTPUT:

In [2]:
```
# Imports and environment check

import numpy as np
import pandas as pd
from sklearn import datasets
from sklearn.model_selection import train_test_split

print("Python version:", __import__("sys").version.split()[0])
print("NumPy version:", np.__version__)
print("Pandas version:", pd.__version__)
import sklearn
print("scikit-learn version:", sklearn.__version__)
```

```
Python version: 3.11.5
NumPy version: 1.24.3
Pandas version: 2.0.3
scikit-learn version: 1.3.0
```

In [3]:
```
# Load the Iris dataset
iris = datasets.load_iris()
X = iris.data           # features (shape 150 x 4)
y = iris.target         # targets (0,1,2)
feature_names = iris.feature_names
target_names = iris.target_names

print("Feature names:", feature_names)
print("Target names:", target_names)
print("Full dataset shape (X):", X.shape)
print("Full target shape (y):", y.shape)
```

```
Feature names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Target names: ['setosa' 'versicolor' 'virginica']
Full dataset shape (X): (150, 4)
Full target shape (y): (150,)
```

Out[4]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	target_name
0	5.1	3.5	1.4	0.2	0	setosa
1	4.9	3.0	1.4	0.2	0	setosa
2	4.7	3.2	1.3	0.2	0	setosa
3	4.6	3.1	1.5	0.2	0	setosa
4	5.0	3.6	1.4	0.2	0	setosa
5	5.4	3.9	1.7	0.4	0	setosa
6	4.6	3.4	1.4	0.3	0	setosa
7	5.0	3.4	1.5	0.2	0	setosa

In [6]:
```
# Split the dataset into 70% train and 30% test
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    train_size=120,   # exact number
    stratify=y,
    random_state=42
)

print("X_train shape:", X_train.shape)
print("X_test shape :", X_test.shape)
print("y_train length:", len(y_train))
print("y_test length :", len(y_test))

# Updated correct assertions
assert X_train.shape[0] == 120, "Training set should contain 120 records"
assert X_test.shape[0] == 30, "Test set should contain 30 records"
```

```
X_train shape: (120, 4)
X_test shape : (30, 4)
y_train length: 120
y_test length : 30
```

```
=== Training set summary ===
(120, 6)
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	target_name
0	4.4	2.9	1.4	0.2	0	setosa
1	4.9	2.5	4.5	1.7	2	virginica
2	6.8	2.8	4.8	1.4	1	versicolor
3	4.9	3.1	1.5	0.1	0	setosa
4	5.5	2.5	4.0	1.3	1	versicolor
...	...	...	...	...	...	...
115	4.9	3.6	1.4	0.1	0	setosa
116	4.7	3.2	1.3	0.2	0	setosa
117	5.5	4.2	1.4	0.2	0	setosa
118	6.9	3.1	4.9	1.5	1	versicolor
119	4.6	3.1	1.5	0.2	0	setosa

120 rows × 6 columns

```
=== Test set summary ===
(30, 6)
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	target_name
0	4.4	3.0	1.3	0.2	0	setosa
1	6.1	3.0	4.9	1.8	2	virginica
2	4.9	2.4	3.3	1.0	1	versicolor
3	5.0	2.3	3.3	1.0	1	versicolor
4	4.4	3.2	1.3	0.2	0	setosa
5	6.3	3.3	4.7	1.6	1	versicolor
6	4.6	3.6	1.0	0.2	0	setosa
7	5.4	3.4	1.7	0.2	0	setosa
8	6.5	3.0	5.2	2.0	2	virginica
9	5.4	3.0	4.5	1.5	1	versicolor
10	7.3	2.9	6.3	1.8	2	virginica
11	6.9	3.1	5.1	2.3	2	virginica
12	6.5	3.0	5.8	2.2	2	virginica
13	6.4	3.2	4.5	1.5	1	versicolor
14	5.0	3.4	1.5	0.2	0	setosa
15	5.0	3.3	1.4	0.2	0	setosa
16	5.8	4.0	1.2	0.2	0	setosa
17	5.6	2.5	3.9	1.1	1	versicolor
18	6.1	2.9	4.7	1.4	1	versicolor
19	6.0	3.0	4.8	1.8	2	virginica
20	5.4	3.7	1.5	0.2	0	setosa
21	6.7	3.1	5.6	2.4	2	virginica

In [8]:
```
# Save to CSV for inspection

df_train.to_csv("iris_train_120.csv", index=False)
df_test.to_csv("iris_test_30.csv", index=False)
print("Saved iris_train_120.csv and iris_test_30.csv in the current notebook directory.")
```

```
Saved iris_train_120.csv and iris_test_30.csv in the current notebook directory.
```

8. Design a perceptron classifier to classify handwritten numerical digits (0-9). Implement using Scikit or Weka.

Introduction:

A perceptron is a simple linear binary classifier. Scikit-learn's Perceptron implements a (multi-class-capable via one-vs-rest) linear model trained with an online/stochastic update rule. Here we will:

- load the digits dataset (0–9),
- preprocess (scale),
- train a Perceptron,
- evaluate with accuracy, classification report and confusion matrix,
- visualize some predictions,
- (optionally) do simple hyperparameter tuning and cross-validation,
- save the model.

Software Requirements:

Python 3.8+

scikit-learn >= 1.0

numpy

matplotlib

Procedure:

Step 1: Imports and notebook setup

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay
# Notebook plot inline (Jupyter)
%matplotlib inline
```

Step 2: Load dataset and inspect

```
digits = load_digits()       # 8x8 images of digits 0-9
X = digits.data              # shape: (n_samples, 64)
y = digits.target            # labels 0-9
print("X shape:", X.shape)
print("y shape:", y.shape)
print("Unique labels:", np.unique(y))
# Show first image and label
plt.figure(figsize=(2.5,2.5))
plt.imshow(digits.images[0], cmap='gray')
plt.title(f"Label: {y[0]}")
plt.axis('off')
```

Step 3: Train/test split

```
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.25, random_state=42, stratify=y
)
print("X_train:", X_train.shape, "X_test:", X_test.shape)
```

Step 4: Scale features

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
print("Mean of scaled training features (approx):", X_train_scaled.mean(axis=0)[:5])
print("Std of scaled training features (approx):", X_train_scaled.std(axis=0)[:5])
```

Step 5: Train a Perceptron

```
clf = Perceptron(max_iter=1000, tol=1e-3, random_state=42, early_stopping=False)
clf.fit(X_train_scaled, y_train)
print("Training complete.")
```

Step 6: Evaluate on test set

```
y_pred = clf.predict(X_test_scaled)
acc = accuracy_score(y_test, y_pred)
print(f"Test accuracy: {acc:.4f}\n")
print("Classification report:")
print(classification_report(y_test, y_pred))
```

```
# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=digits.target_names)
fig, ax = plt.subplots(figsize=(8,6))
disp.plot(ax=ax, cmap='Blues')
plt.title("Confusion Matrix (Perceptron)")
plt.show()
```

Step 7: Visualize some predictions

```
n_show = 12
plt.figure(figsize=(12, 4))
for i in range(n_show):
    ax = plt.subplot(2, 6, i+1)
    img = X_test[i].reshape(8, 8)
    ax.imshow(img, cmap='gray')
    ax.set_title(f"T:{y_test[i]} P:{y_pred[i]}")
    ax.axis('off')
plt.suptitle("True (T) vs Predicted (P)")
plt.show()
```

Step 8: Cross-validation

```
X_scaled_all = scaler.fit_transform(X)   # scale using whole data for CV demonstration
perceptron_cv = Perceptron(max_iter=1000, tol=1e-3, random_state=42)
cv_scores = cross_val_score(perceptron_cv, X_scaled_all, y, cv=5, scoring='accuracy')
print("5-fold CV accuracies:", np.round(cv_scores, 4))
print("Mean CV accuracy:", np.mean(cv_scores).round(4))
```

Step 9: Simple hyperparameter tuning

```
param_grid = {
    'alpha': [0.00001, 0.0001, 0.001, 0.01],
    'max_iter': [500, 1000, 2000]
}
gs = GridSearchCV(Perceptron(random_state=42), param_grid, cv=4, scoring='accuracy', n_jobs=-
1)
gs.fit(X_train_scaled, y_train)
print("Best params:", gs.best_params_)
```

print("Best CV score (on training set folds):", gs.best_score_)

best = gs.best_estimator_

print("Test accuracy of best estimator:", accuracy_score(y_test, best.predict(X_test_scaled)))

Step 10: Save scaler and model

import joblib

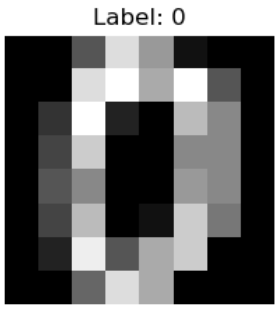
joblib.dump(scaler, "digits_scaler.joblib")

joblib.dump(clf, "perceptron_digits_model.joblib")

print("Saved scaler and model to files.")

OUTPUT:

```
In [4]: # Load dataset and inspect
        digits = load_digits()        # 8x8 images of digits 0-9
        X = digits.data               # shape: (n_samples, 64)
        y = digits.target             # labels 0-9

        print("X shape:", X.shape)
        print("y shape:", y.shape)
        print("Unique labels:", np.unique(y))

        # Show first image and label
        plt.figure(figsize=(2.5,2.5))
        plt.imshow(digits.images[0], cmap='gray')
        plt.title(f"Label: {y[0]}")
        plt.axis('off')

        X shape: (1797, 64)
        y shape: (1797,)
        Unique labels: [0 1 2 3 4 5 6 7 8 9]

Out[4]: (-0.5, 7.5, 7.5, -0.5)
```



In [5]:
```
# Train/test split
# We'll hold out 25% for test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

print("X_train:", X_train.shape, "X_test:", X_test.shape)
```

```
X_train: (1347, 64) X_test: (450, 64)
```

In [7]:
```
# Step 4: Scale features
# Perceptron is a linear model — scaling helps convergence
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)

# Quick check
print("Mean of scaled training features (approx):", X_train_scaled.mean(axis=0)[:5])
print("Std of scaled training features (approx):", X_train_scaled.std(axis=0)[:5])
```

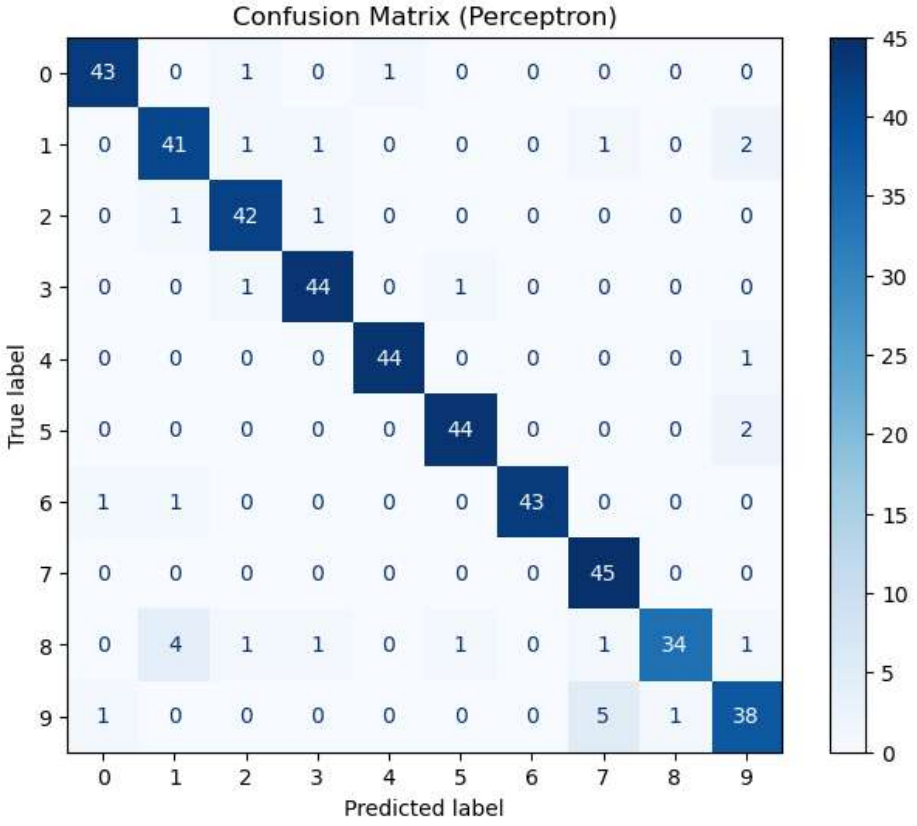
```
Mean of scaled training features (approx): [ 0.00000000e+00 -8.43176061e-17  8.16801052e-17 -2.03252411e-16
  1.10816248e-16]
Std of scaled training features (approx): [0. 1. 1. 1. 1.]
```

In [8]:
```
# Train a Perceptron
# We'll set a max_iter and tol for convergence and a fixed random_state for reproducibility.
clf = Perceptron(max_iter=1000, tol=1e-3, random_state=42, early_stopping=False)
clf.fit(X_train_scaled, y_train)

print("Training complete.")
```

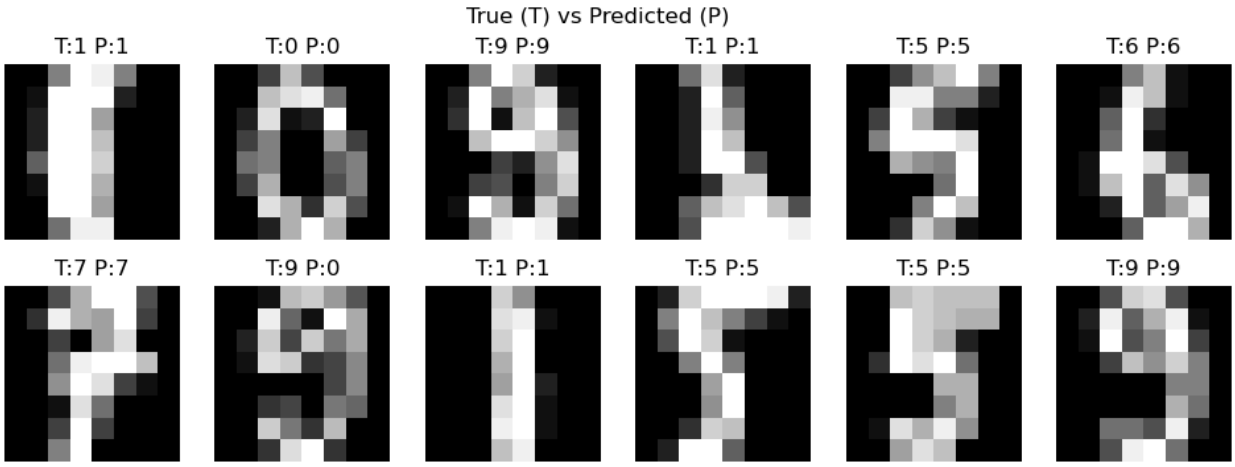
```
Training complete.
```

In [9]:
```
# Evaluate on test set
y_pred = clf.predict(X_test_scaled)

acc = accuracy_score(y_test, y_pred)
print(f"Test accuracy: {acc:.4f}\n")

print("Classification report:")
print(classification_report(y_test, y_pred))

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=digits.target_names)
fig, ax = plt.subplots(figsize=(8,6))
disp.plot(ax=ax, cmap='Blues')
plt.title("Confusion Matrix (Perceptron)")
plt.show()
```

```
Test accuracy: 0.9289

Classification report:
              precision    recall  f1-score   support

           0       0.96      0.96      0.96        45
           1       0.87      0.89      0.88        46
           2       0.91      0.95      0.93        44
           3       0.94      0.96      0.95        46
           4       0.98      0.98      0.98        45
           5       0.96      0.96      0.96        46
           6       1.00      0.96      0.98        45
           7       0.87      1.00      0.93        45
           8       0.97      0.79      0.87        43
           9       0.86      0.84      0.85        45

    accuracy                           0.93       450
   macro avg       0.93      0.93      0.93       450
weighted avg       0.93      0.93      0.93       450
```



In [10]:
```
# Visualize some predictions
# show first 12 test samples with predicted vs true labels
n_show = 12
plt.figure(figsize=(12, 4))
for i in range(n_show):
    ax = plt.subplot(2, 6, i+1)
    img = X_test[i].reshape(8, 8)
    ax.imshow(img, cmap='gray')
    ax.set_title(f"T:{y_test[i]} P:{y_pred[i]}")
    ax.axis('off')
plt.suptitle("True (T) vs Predicted (P)")
plt.show()
```



In [11]:
```
# Cross-validation
# 5-fold CV on the whole dataset (scaled) to get a stable estimate
X_scaled_all = scaler.fit_transform(X)   # scale using whole data for CV demonstration
perceptron_cv = Perceptron(max_iter=1000, tol=1e-3, random_state=42)
cv_scores = cross_val_score(perceptron_cv, X_scaled_all, y, cv=5, scoring='accuracy')
print("5-fold CV accuracies:", np.round(cv_scores, 4))
print("Mean CV accuracy:", np.mean(cv_scores).round(4))
```

```
5-fold CV accuracies: [0.8667 0.8361 0.9053 0.9443 0.8552]
Mean CV accuracy: 0.8815
```

In [12]:
```
# Simple hyperparameter tuning
# We'll tune 'alpha' (L2 penalty) and 'max_iter' using GridSearchCV
param_grid = {
    'alpha': [0.00001, 0.0001, 0.001, 0.01],
    'max_iter': [500, 1000, 2000]
}
# Note: Perceptron accepts 'alpha' parameter as L2 regularization via parameter name in older sklearn versions.
# If sklearn raises an error for 'alpha', remove alpha from grid and tune max_iter only.
gs = GridSearchCV(Perceptron(random_state=42), param_grid, cv=4, scoring='accuracy', n_jobs=-1)
gs.fit(X_train_scaled, y_train)
print("Best params:", gs.best_params_)
print("Best CV score (on training set folds):", gs.best_score_)
best = gs.best_estimator_
print("Test accuracy of best estimator:", accuracy_score(y_test, best.predict(X_test_scaled)))
```

```
Best params: {'alpha': 1e-05, 'max_iter': 500}
Best CV score (on training set folds): 0.9316845061466723
Test accuracy of best estimator: 0.9288888888888889
```

In [13]:
```
# Save scaler and model
import joblib
joblib.dump(scaler, "digits_scaler.joblib")
joblib.dump(clf, "perceptron_digits_model.joblib")
print("Saved scaler and model to files.")
```

```
Saved scaler and model to files.
```

10. NLP: Program to illustrate the concepts sentence segmentation, word tokenization, stemming and lemmatization, Hidden markov model (HMM) for Parts of speech (PoS)tag.

Introduction:

Natural Language Processing (NLP) is a branch of artificial intelligence that enables computers to understand, interpret, and generate human language. It focuses on processing large amounts of natural text to perform tasks such as translation, summarization, sentiment analysis, and speech recognition. Before any advanced NLP task, text must be cleaned and prepared through preprocessing steps.

Sentence segmentation is the process of splitting a paragraph or document into individual sentences. It helps the system understand where ideas begin and end.

Tokenization breaks sentences into smaller units called tokens, usually words or subwords. This step is essential for feeding text into machine learning or deep learning models.

Stemming reduces words to their root form by chopping off suffixes (e.g., "running" → "run"), though the result may not be a real word.

Lemmatization reduces words to their meaningful dictionary form by considering grammar and context (e.g., "better" → "good").

These preprocessing steps improve the accuracy and efficiency of NLP models.

Procedure:

Step 1: Imports and NLTK data download

```
import nltk
from nltk import sent_tokenize, word_tokenize
from nltk.corpus import treebank, wordnet
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer
from nltk.tag import hmm
import random
import pprint
nltk.download('punkt')        # sentence & word tokenizer
nltk.download('averaged_perceptron_tagger')  # not strictly needed for HMM but useful
nltk.download('treebank')      # tagged sentences for training
nltk.download('wordnet')       # lemmatizer corpus
nltk.download('omw-1.4')       # lemmatizer extra data
```

Step 2: Sentence segmentation

```
raw_text = """
Natural Language Processing (NLP) is a field of artificial intelligence.
It focuses on the interaction between computers and human (natural) languages.
Let's illustrate sentence segmentation, tokenization, stemming, lemmatization, and HMM POS tagging.
"""
sentences = sent_tokenize(raw_text)
print("Segmented sentences:")
for i, s in enumerate(sentences, 1):
    print(i, s)
```

Step 3: Word tokenization

```
tokenized_sentences = [word_tokenize(sent) for sent in sentences]
print("\nTokenized sentences (words):")
pprint.pprint(tokenized_sentences)
```

Step 4: Stemming and Lemmatization

```
ps = PorterStemmer()
wnl = WordNetLemmatizer()
def lemmatize_word(w):
    # Simple lemmatization: default pos='n' -> try a small heuristic
    # For demonstration we just call with default noun; advanced use: map POS tags to wordnet POS
    return wnl.lemmatize(w)
print("\nStemming and Lemmatization results (per token in first sentence):")
for token in tokenized_sentences[0]:
    stem = ps.stem(token)
    lemma = lemmatize_word(token)
    print(f"{token:15} -> stem: {stem:10} | lemma: {lemma}")
```

OUTPUT:

```
Out[3]: True
```

```
In [4]: # Sentence segmentation
raw_text = """
Natural Language Processing (NLP) is a field of artificial intelligence.
It focuses on the interaction between computers and human (natural) languages.
Let's illustrate sentence segmentation, tokenization, stemming, lemmatization, and HMM POS tagging.
"""

sentences = sent_tokenize(raw_text)
print("Segmented sentences:")
for i, s in enumerate(sentences, 1):
    print(i, s)
```

```
Segmented sentences:
1
Natural Language Processing (NLP) is a field of artificial intelligence.
2 It focuses on the interaction between computers and human (natural) languages.
3 Let's illustrate sentence segmentation, tokenization, stemming, lemmatization, and HMM POS tagging.
```

```
In [5]: # Word tokenization
tokenized_sentences = [word_tokenize(sent) for sent in sentences]
print("\nTokenized sentences (words):")
pprint.pprint(tokenized_sentences)
```

```
Tokenized sentences (words):
[['Natural',
  'Language',
  'Processing',
  '(',
  'NLP',
  ')',
  'is',
  'a',
  'field',
  'of',
  'artificial',
  'intelligence',
  '.'],
 ['It',
  'focuses',
  'on',
  'the',
  'interaction',
  'between',
  'computers',
  'and',
  'human',
  '(',
  'natural',
  ')',
  'languages',
  '.'],
 ['Let',
  "'s",
  'illustrate',
  'sentence',
  'segmentation',
  ',',
```

In [6]:
```
# Stemming and Lemmatization
ps = PorterStemmer()
wnl = WordNetLemmatizer()

def lemmatize_word(w):
    # Simple lemmatization: default pos='n' -> try a small heuristic
    # For demonstration we just call with default noun; advanced use: map POS tags to wordnet POS
    return wnl.lemmatize(w)

print("\nStemming and Lemmatization results (per token in first sentence):")
for token in tokenized_sentences[0]:
    stem = ps.stem(token)
    lemma = lemmatize_word(token)
    print(f"{token:15} -> stem: {stem:10} | lemma: {lemma}")
```

```
Stemming and Lemmatization results (per token in first sentence):
Natural         -> stem: natur      | lemma: Natural
Language        -> stem: languag    | lemma: Language
Processing      -> stem: process    | lemma: Processing
(               -> stem: (          | lemma: (
NLP             -> stem: nlp        | lemma: NLP
)               -> stem: )          | lemma: )
is              -> stem: is         | lemma: is
a               -> stem: a          | lemma: a
field           -> stem: field      | lemma: field
of              -> stem: of         | lemma: of
artificial      -> stem: artifici   | lemma: artificial
intelligence    -> stem: intellig   | lemma: intelligence
.               -> stem: .          | lemma: .
```